

Java 8 — Lambda Expressions

Java Version: 8

Year: 2014

Feature: Lambda Expressions

UniqueClassName: LambdaExpressions

1. Definition

A **lambda expression** in Java is a concise way to represent an **anonymous function**—a block of executable code that can be passed as a value.

Lambda expressions were introduced in **Java 8** primarily to support **functional programming** and make it easier to work with **functional interfaces**, collections, and the Stream API.

General Syntax

```
(parameters) -> expression
```

or

```
(parameters) -> {  
    // statements  
}
```

Examples

```
() -> System.out.println("Hello");
```

```
x -> x * x
```

```
(a, b) -> a + b
```

```
(a, b) -> {  
    int result = a + b;  
    return result;  
}
```

2. Why Lambda Expressions Were Introduced

Before Java 8, implementing behavior generally required an **anonymous inner class**.

Before Java 8

```
Runnable r = new Runnable() {  
    @Override  
    public void run() {  
        System.out.println("Hello");  
    }  
};
```

```
}  
};
```

Java 8

```
Runnable r = () -> System.out.println("Hello");
```

The lambda eliminates the boilerplate associated with the anonymous class.

3. Important Rule: Lambda Requires a Functional Interface

A lambda expression does **not** exist independently. It must be assigned to a **functional interface**.

A functional interface is an interface containing **exactly one abstract method**.

For example:

```
@FunctionalInterface  
interface Calculator {  
    int calculate(int a, int b);  
}
```

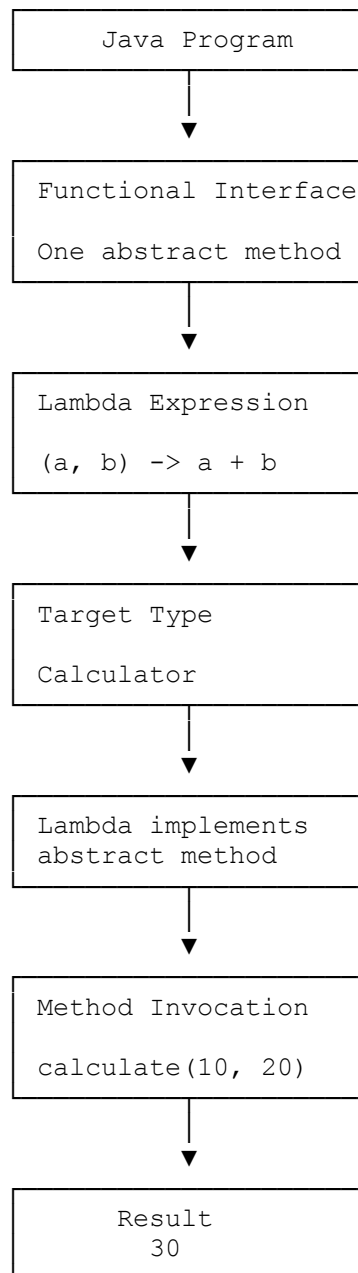
Now a lambda can implement that method:

```
Calculator c = (a, b) -> a + b;
```

Conceptually:

```
Calculator interface  
    |  
    | calculate(int, int)  
    |  
    v  
Lambda Expression  
    |  
    | (a, b) -> a + b  
    v  
Implementation of calculate()
```

4. Lambda Expression Flowchart



5. Java Code Example

```
@FunctionalInterface
interface Calculator {
    int calculate(int a, int b);
}

public class LambdaExpressions {
```

```
public static void main(String[] args) {  
  
    // Lambda expression  
    Calculator addition = (a, b) -> a + b;  
  
    // Calling the functional interface method  
    int result = addition.calculate(10, 20);  
  
    System.out.println("Result = " + result);  
}  
}
```

Output

Result = 30

6. Execution Flow

For:

Calculator addition = (a, b) -> a + b;

the logical sequence is:

1. Calculator interface is identified
↓
 2. calculate(int, int) is its single abstract method
↓
 3. Lambda (a, b) -> a + b is assigned to Calculator
↓
 4. Lambda becomes the implementation of calculate()
↓
 5. addition.calculate(10, 20) is called
↓
 6. a = 10, b = 20
↓
 7. a + b is evaluated
↓
 8. 30 is returned
↓
 9. result receives 30
-

7. Lambda Expression Components

Consider:

`(a, b) -> a + b`

Component	Meaning
(a, b)	Parameters
->	Lambda operator
a + b	Lambda body
a + b	Return expression
Target interface	Determines the parameter and return types

For example:

Calculator addition = `(a, b) -> a + b;`

Because Calculator declares:

```
int calculate(int a, int b);
```

Java understands that a and b are int parameters and the lambda returns an int.

8. Different Lambda Syntax Forms

No parameters

```
Runnable r = () -> System.out.println("Hello");
```

One parameter

```
Consumer<String> c = name -> System.out.println(name);
```

Parentheses are optional for a single implicitly typed parameter.

Both are possible:

```
name -> System.out.println(name);  
(name) -> System.out.println(name);
```

Multiple parameters

```
BinaryOperator<Integer> add = (a, b) -> a + b;
```

Multiple statements

```
Calculator calculator = (a, b) -> {  
    int result = a + b;  
    return result;  
};
```

9. Lambda with Java's Built-in Functional Interfaces

Java 8 provides many functional interfaces in `java.util.function`.

Predicate<T>

Returns boolean.

```
Predicate<Integer> even = n -> n % 2 == 0;
```

```
System.out.println(even.test(10));
```

Output:

true

Function<T, R>

Accepts one value and returns another value.

```
Function<Integer, Integer> square = n -> n * n;
```

```
System.out.println(square.apply(5));
```

Output:

25

Consumer<T>

Accepts a value but returns nothing.

```
Consumer<String> print = name -> System.out.println(name);
```

```
print.accept("Java");
```

Output:

Java

Supplier<T>

Produces a value without accepting an argument.

```
Supplier<String> message = () -> "Hello Java";
```

```
System.out.println(message.get());
```

Output:

Hello Java

10. Lambda with Collections

One of the major benefits of lambda expressions is simpler collection processing.

```
import java.util.Arrays;
import java.util.List;

public class LambdaExpressions {

    public static void main(String[] args) {

        List<String> names =
            Arrays.asList("Anurag", "Rahul", "Amit", "Vijay");

        names.forEach(name -> System.out.println(name));
    }
}
```

Output:

Anurag
Rahul
Amit
Vijay

Here:

```
name -> System.out.println(name)
```

is passed as behavior to:

forEach()

11. Lambda with Stream API

Lambda expressions became especially important with the Java 8 Stream API.

```
import java.util.Arrays;
import java.util.List;

public class LambdaExpressions {

    public static void main(String[] args) {

        List<Integer> numbers =
            Arrays.asList(10, 15, 20, 25, 30);

        numbers.stream()
            .filter(n -> n % 2 == 0)
            .forEach(n -> System.out.println(n));
    }
}
```

Output:

```
10
20
30
```

The flow is:

```
List
↓
stream()
↓
filter(n -> n % 2 == 0)
↓
Keep even numbers
↓
forEach(n -> System.out.println(n))
↓
10
20
30
```

12. Lambda vs Anonymous Inner Class

Feature	Anonymous Inner Class	Lambda Expression
Introduced	Before Java 8	Java 8
Syntax	Verbose	Concise
Main purpose	Implement interface/class behavior	Implement functional-interface behavior
Requires functional interface	No	Yes
this reference	Refers to anonymous class object	Refers to enclosing object
Boilerplate	More	Less
Common usage	Legacy APIs, multiple methods	Streams, collections, callbacks
Readability	Usually lower	Usually higher

Anonymous class

```
Runnable r = new Runnable() {  
    @Override  
    public void run() {  
        System.out.println("Running");  
    }  
};
```

Lambda

```
Runnable r = () -> System.out.println("Running");
```

13. Key Characteristics

1. **Introduced in Java 8 (2014).**
2. Provides a concise representation of behavior.
3. Works with **functional interfaces**.
4. Uses the -> operator.
5. Supports functional programming.
6. Reduces anonymous-class boilerplate.

7. Works extensively with the **Stream API**.
8. Can be passed as an argument to methods.
9. Can be stored in variables whose target type is a functional interface.
10. Can return values or perform actions.

Core Concept

Lambda Expression
↓
Functional Interface
↓
Implementation of its
single abstract method
↓
Method invocation
↓
Result / Action

In one sentence: A Java 8 lambda expression is a compact way of supplying the implementation of a functional interface's single abstract method, enabling behavior to be treated as a value and passed around more easily.

Java 8 — Functional Interfaces

Java Version: 8

Year: 2014

Feature: Functional Interfaces

UniqueClassName: FunctionalInterfaces

1. Definition

A **functional interface** is an interface that contains **exactly one abstract method**.

It may contain:

- Exactly **one abstract method**
- Any number of default methods
- Any number of static methods
- Methods inherited from Object do **not** count as abstract methods for this purpose

Functional interfaces are the **target types for lambda expressions and method references** introduced/enhanced in Java 8.

Basic Syntax

```
@FunctionalInterface
interface InterfaceName {

    // Exactly one abstract method
    returnType methodName(parameters);

    // Optional
    default void defaultMethod() {
        // implementation
    }

    // Optional
    static void staticMethod() {
        // implementation
    }
}
```

2. Simple Example

```
@FunctionalInterface
interface Calculator {

    int add(int a, int b);
}
```

Calculator is a functional interface because it has exactly **one abstract method**:

```
int add(int a, int b);
```

A lambda expression can provide its implementation:

```
Calculator calculator = (a, b) -> a + b;
```

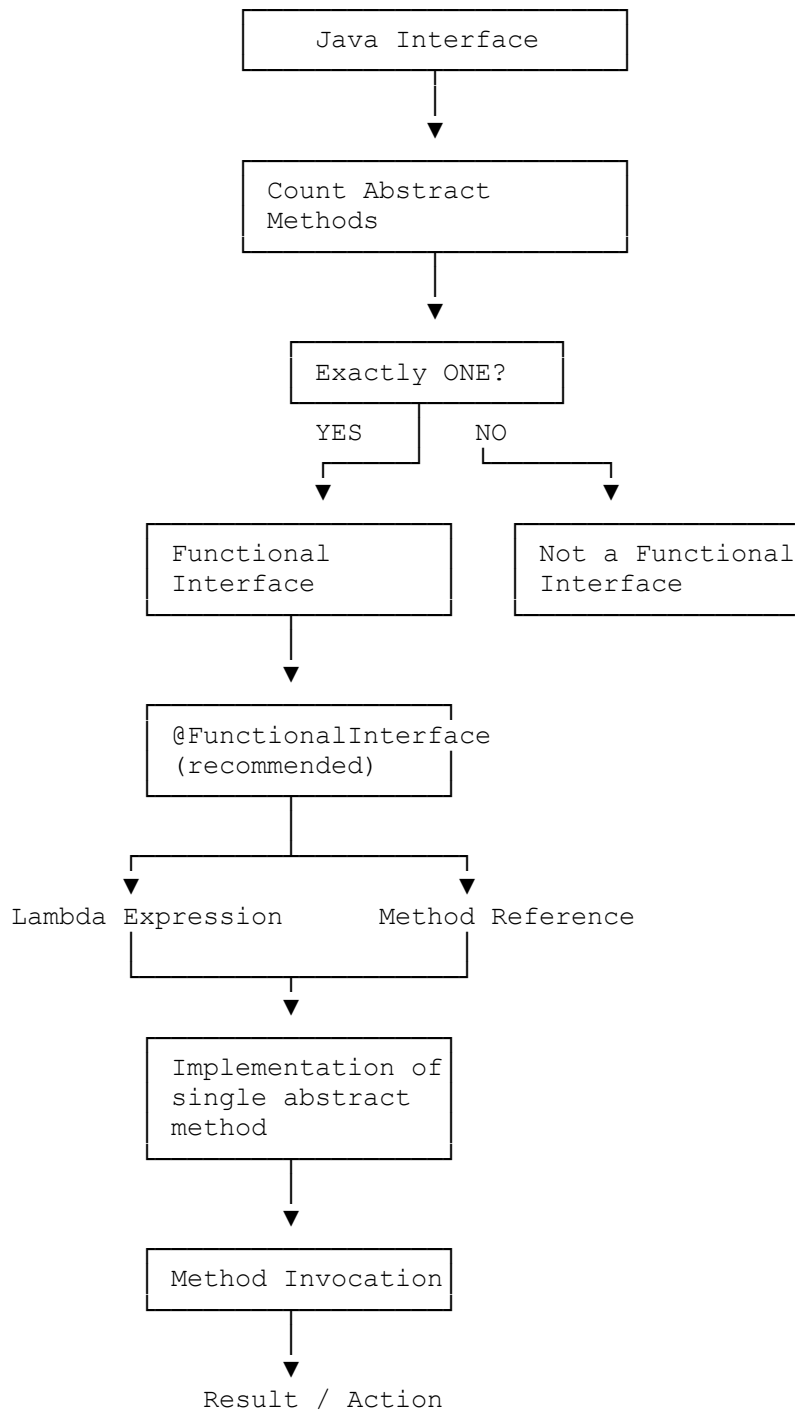
Then:

```
int result = calculator.add(10, 20);
```

produces:

30

3. Functional Interface Flowchart



4. Complete Java Code Example

```
@FunctionalInterface
interface Calculator {

    // Single abstract method
    int calculate(int a, int b);

    // Allowed: default method
    default void showMessage() {
        System.out.println("Calculator is ready.");
    }

    // Allowed: static method
    static void info() {
        System.out.println("This is a functional interface.");
    }
}

public class FunctionalInterfaces {

    public static void main(String[] args) {

        // Lambda implements the single abstract method
        Calculator addition = (a, b) -> a + b;

        // Calling default method
        addition.showMessage();

        // Calling abstract method implementation
        int result = addition.calculate(10, 20);

        // Calling static interface method
        Calculator.info();

        System.out.println("Result = " + result);
    }
}
```

Output

Calculator is ready.
This is a functional interface.
Result = 30

5. How the Code Works

The interface contains:

```
int calculate(int a, int b);
```

This is the **only abstract method**.

Therefore:

```
@FunctionalInterface  
interface Calculator
```

is a valid functional interface.

The lambda:

$(a, b) \rightarrow a + b$

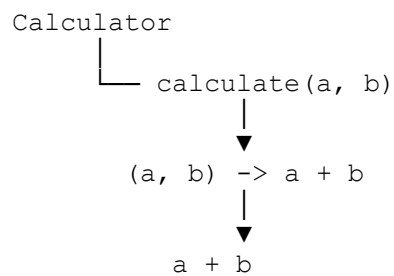
provides the implementation of:

`calculate(int a, int b)`

So:

`Calculator addition = (a, b) -> a + b;`

can be conceptually understood as:



When this executes:

`addition.calculate(10, 20);`

the values are:

`a = 10`

`b = 20`

`10 + 20`

$\downarrow$
`30`

6. What Does @FunctionalInterface Do?

@FunctionalInterface is an annotation used to explicitly declare that an interface is intended to be a functional interface.

Example:

```
@FunctionalInterface
interface Calculator {
    int calculate(int a, int b);
}
```

The compiler verifies the rule.

If you accidentally add another abstract method:

```
@FunctionalInterface
interface Calculator {

    int calculate(int a, int b);

    int subtract(int a, int b);
}
```

the compiler reports an error because the interface now has **two abstract methods**.

Therefore:

1 abstract method → Valid functional interface
2+ abstract methods → Invalid functional interface

The annotation is not strictly required for an interface to qualify structurally as functional, but it is strongly recommended because it allows the compiler to enforce the intended design.

7. Default and Static Methods Do Not Break the Rule

Consider:

```
@FunctionalInterface
interface Calculator {

    int calculate(int a, int b);           // Abstract

    default void display() {               // Not abstract
        System.out.println("Display");
    }

    static void information() {            // Not abstract
    }
```

```

        System.out.println("Info");
    }
}

```

Abstract-method count:

```

calculate()      → 1 abstract method
display()       → default method
information()    → static method

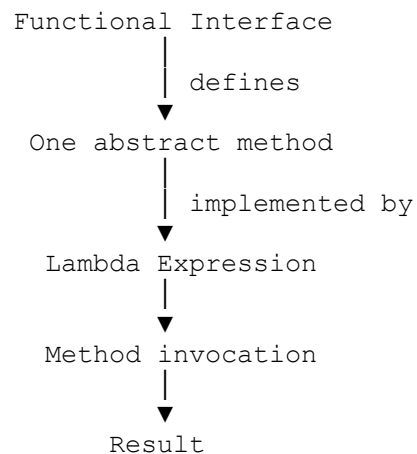
```

Total abstract methods = 1

Therefore it is still a functional interface.

8. Functional Interface + Lambda Relationship

The most important relationship is:



For example:

```

@FunctionalInterface
interface Square {
    int calculate(int number);
}

```

Lambda:

Square square = n -> n * n;

Invocation:

```
System.out.println(square.calculate(5));
```

Output:

25

9. Built-in Functional Interfaces in Java 8

Java provides many functional interfaces in `java.util.function`.

Functional Interface	Abstract Method	Purpose	Example
Predicate<T>	<code>boolean test(T)</code>	Tests a condition	<code>n -> n > 10</code>
Function<T,R>	<code>R apply(T)</code>	Converts one value to another	<code>n -> n * 2</code>
Consumer<T>	<code>void accept(T)</code>	Consumes a value	<code>n -> System.out.println(n)</code>
Supplier<T>	<code>T get()</code>	Supplies a value	<code>() -> "Java"</code>
UnaryOperator<T>	<code>T apply(T)</code>	Same type → same type	<code>n -> n * n</code>
BinaryOperator<T>	<code>T apply(T,T)</code>	Two same-type values → same type	<code>(a,b) -> a+b</code>

10. Example Using Predicate

```
import java.util.function.Predicate;

public class FunctionalInterfaces {

    public static void main(String[] args) {

        Predicate<Integer> evenNumber =
            number -> number % 2 == 0;

        System.out.println(evenNumber.test(10));
        System.out.println(evenNumber.test(15));
    }
}
```

Output:

true
false

Here:

Predicate<Integer>

is a functional interface whose single abstract method is:

boolean test(Integer t);

The lambda:

number -> number % 2 == 0

implements that method.

11. Functional Interface vs Normal Interface

Feature	Normal Interface	Functional Interface
Abstract methods	Can have multiple	Exactly one
Lambda target	Not necessarily	Yes
Method reference target	Not necessarily	Yes
default methods	Allowed	Allowed
static methods	Allowed	Allowed
@FunctionalInterface	Not applicable/necessary	Recommended
Main purpose	Define general contract	Represent one unit of behavior

12. Important Examples of Functional Interfaces

Custom functional interface

```
@FunctionalInterface
interface Greeting {
    void sayHello(String name);
}
```

Lambda implementation

```
Greeting greeting =
    name -> System.out.println("Hello " + name);
```

Invocation

```
greeting.sayHello("Anurag");
```

Output:

Hello Anurag

13. Functional Interface and Method Reference

Functional interfaces also provide the target type for **method references**.

```
@FunctionalInterface
interface Printer {
    void print(String message);
}
```

Using a lambda:

```
Printer p = message -> System.out.println(message);
```

Using a method reference:

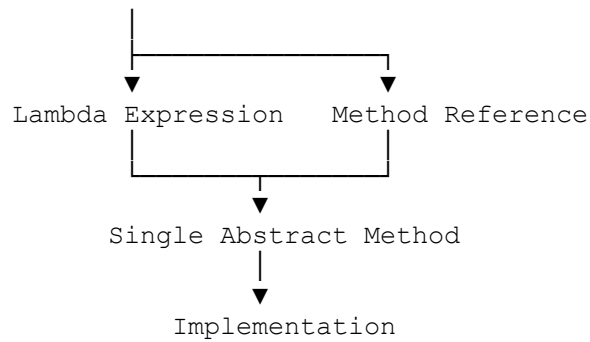
```
Printer p = System.out::println;
```

Both represent an implementation of:

```
void print(String message);
```

The relationship is:

Functional Interface



14. Key Rules to Remember

Functional Interface

- Exactly ONE abstract method
- Can have multiple default methods
- Can have multiple static methods
- Can be implemented using lambda
- Can be target of method reference
- `@FunctionalInterface` helps compiler verify it

One-line definition

A functional interface in Java is an interface with exactly one abstract method, which can serve as the target type for a lambda expression or method reference.

Java 8 — Streams API

Java Version: 8

Year: 2014

Feature: Streams API

UniqueClassName: StreamsAPI

1. Definition

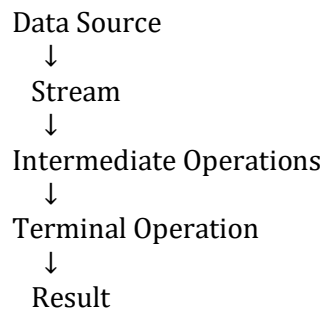
The **Streams API** was introduced in **Java 8** to process collections and other data sources in a **declarative, functional, and pipeline-based manner**.

A **Stream** is a sequence of elements that can be processed through a pipeline of operations such as:

- filtering
- mapping
- sorting
- transforming
- reducing
- collecting

A Stream **does not store data**. Instead, it processes data from a source such as a Collection, array, or I/O channel.

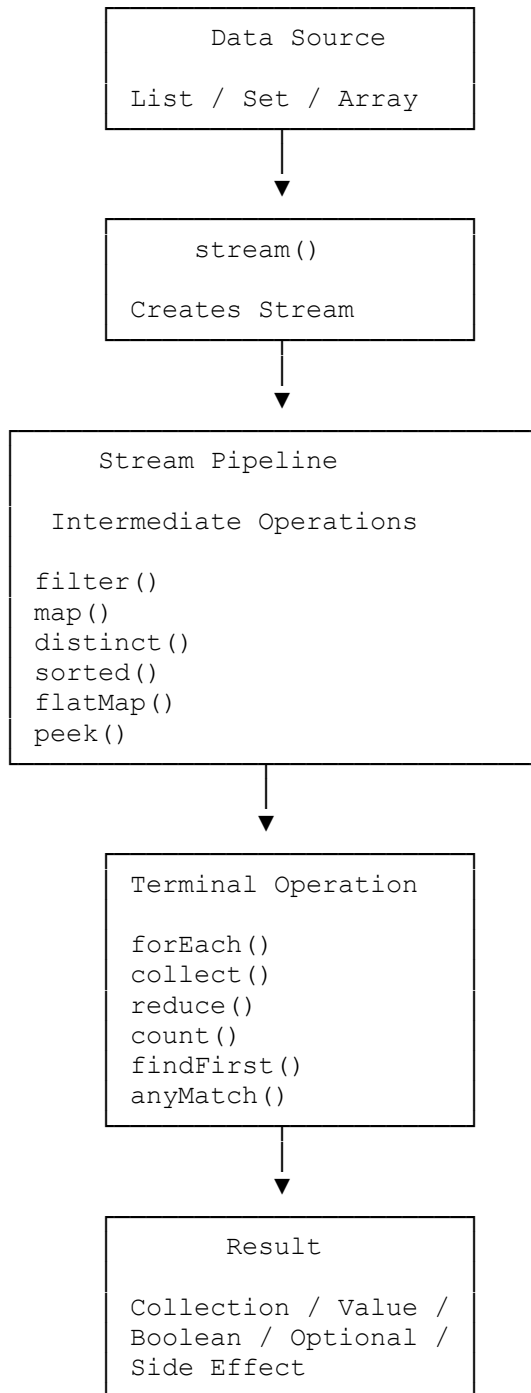
Basic concept



For example:

```
numbers.stream()
    .filter(n -> n % 2 == 0)
    .map(n -> n * 2)
    .forEach(System.out::println);
```

2. Streams API Flowchart



3. Three Main Parts of a Stream Pipeline

A Stream pipeline normally consists of three parts:

Source → Intermediate Operations → Terminal Operation

For example:

```
numbers.stream()
    .filter(n -> n % 2 == 0)
    .map(n -> n * 2)
    .collect(Collectors.toList());
```

Part 1 — Source

```
numbers.stream()
```

Creates a Stream from the collection.

Part 2 — Intermediate operations

```
.filter(n -> n % 2 == 0)
.map(n -> n * 2)
```

These transform/process the stream.

Part 3 — Terminal operation

```
.collect(Collectors.toList())
```

Terminates the pipeline and produces a result.

4. Complete Java Code Example

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class StreamsAPI {

    public static void main(String[] args) {

        // Source
        List<Integer> numbers =
            Arrays.asList(10, 15, 20, 25, 30, 35, 40);

        // Stream pipeline
        List<Integer> result = numbers.stream()
```

```

        // Intermediate operation
        .filter(n -> n % 2 == 0)

        // Intermediate operation
        .map(n -> n * 2)

        // Terminal operation
        .collect(Collectors.toList());

    System.out.println(result);
}
}

```

Output

[20, 40, 60, 80]

5. How the Example Executes

Original collection:

[10, 15, 20, 25, 30, 35, 40]

Step 1 — Create Stream

```

numbers.stream()
List
↓
Stream

```

Step 2 — filter()

```
.filter(n -> n % 2 == 0)
```

Only even numbers survive:

10	15	20	25	30	35	40
↓		↓		↓		↓
10		20		30		40

Step 3 — map()

```
.map(n -> n * 2)
```

Each remaining element is transformed:

10 → 20
20 → 40
30 → 60
40 → 80

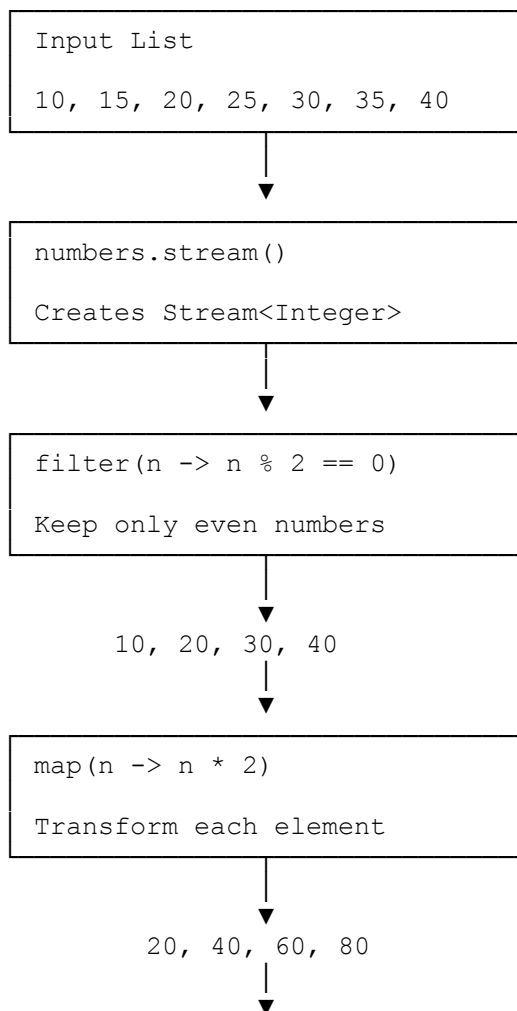
Step 4 — collect()

```
.collect(Collectors.toList())
```

The processed elements are collected into a List:

[20, 40, 60, 80]

6. Complete Execution Flow



```
collect(Collectors.toList())  
Stream → List
```

↓
[20, 40, 60, 80]

7. Intermediate Operations

Intermediate operations return another Stream, allowing operations to be chained.

Operation	Purpose	Example
filter()	Select elements	filter(n -> n > 10)
map()	Transform elements	map(n -> n * 2)
flatMap()	Flatten nested streams	flatMap(List::stream)
distinct()	Remove duplicates	distinct()
sorted()	Sort elements	sorted()
limit()	Limit number of elements	limit(5)
skip()	Skip elements	skip(2)
peek()	Observe elements	peek(System.out::println)

Example:

```
numbers.stream()  
    .filter(n -> n > 10)  
    .distinct()  
    .sorted()  
    .limit(5);
```

Important: Intermediate operations are generally **lazy**. They do not actually process the elements until a terminal operation is invoked.

8. Terminal Operations

Terminal operations end the Stream pipeline.

Operation	Purpose
forEach()	Perform action on each element
collect()	Collect elements into a result
reduce()	Combine elements into one result
count()	Count elements
min()	Find minimum
max()	Find maximum
findFirst()	Find first element
findAny()	Find any element
anyMatch()	Test whether any element matches
allMatch()	Test whether all elements match
noneMatch()	Test whether no elements match

Example:

```
long count = numbers.stream()
                    .filter(n -> n % 2 == 0)
                    .count();
```

Result:

4

9. Stream Operations Are Usually Lazy

Consider:

```
numbers.stream()
        .filter(n -> {
            System.out.println("Filtering: " + n);
```

```
        return n > 20;
    })
    .map(n -> n * 2);
```

Nothing needs to be consumed yet because there is **no terminal operation**.

Add:

```
.forEach(System.out::println);
```

Now the pipeline is executed.

Source

↓

filter()

↓

map()

↓

forEach()

↓

Execution

This is one of the fundamental characteristics of the Streams API.

10. Stream Does Not Modify the Original Collection

Example:

```
List<Integer> numbers =
    Arrays.asList(10, 20, 30, 40);
```

```
List<Integer> result = numbers.stream()
    .map(n -> n * 2)
    .collect(Collectors.toList());
```

The original list remains:

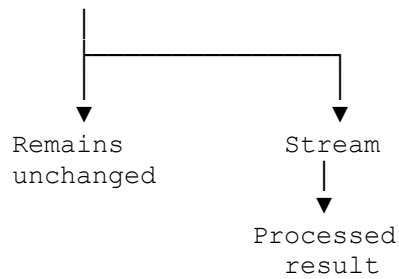
```
numbers = [10, 20, 30, 40]
```

The new result is:

```
result = [20, 40, 60, 80]
```

Therefore:

Original Data



11. Sequential Stream vs Parallel Stream

Java 8 also provides parallel stream processing.

Sequential

```
numbers.stream()  
    .filter(n -> n % 2 == 0)  
    .forEach(System.out::println);
```

Parallel

```
numbers.parallelStream()  
    .filter(n -> n % 2 == 0)  
    .forEach(System.out::println);
```

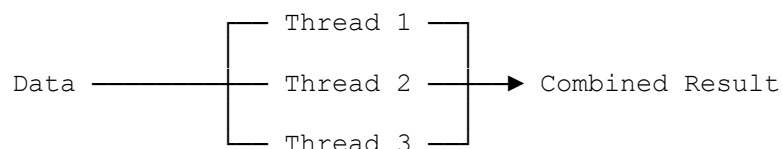
Conceptually:

Sequential Stream

Data
↓
Thread
↓
Process
↓
Result

versus:

Parallel Stream



Parallel streams can be useful for suitable CPU-intensive workloads, but they are **not automatically faster** than sequential streams.

12. Stream vs Collection

Collection	Stream
Stores data	Processes data
Data structure	Processing abstraction
Can be reused	A stream is generally single-use
Supports adding/removing elements depending on type	Does not store elements
Focuses on data	Focuses on computation
Example: List	Example: Stream

A useful mental model is:

Collection = "What data do I have?"

Stream = "What do I want to do with that data?"

13. Stream Pipeline Example with Strings

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class StreamsAPI {

    public static void main(String[] args) {

        List<String> names =
            Arrays.asList(
                "Anurag",
                "Amit",
                "Rahul",
                "Anil",
                "Vijay"
            );

        List<String> result = names.stream()
```

```

        .filter(name -> name.startsWith("A"))
        .map(String::toUpperCase)
        .sorted()
        .collect(Collectors.toList());

    System.out.println(result);
}
}

```

Output:

[AMIT, ANIL, ANURAG]

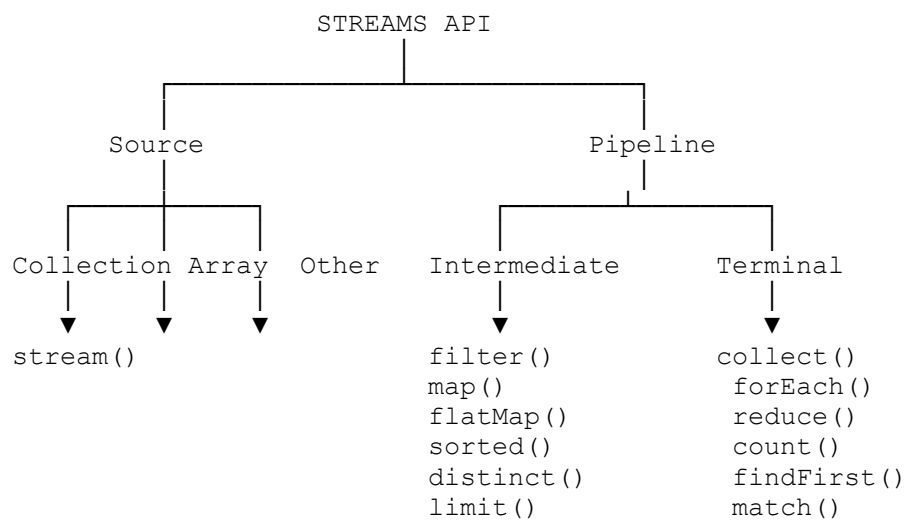
Flow:

```

Names
↓
stream()
↓
filter(startsWith("A"))
↓
map(toUpperCase)
↓
sorted()
↓
collect(toList())
↓
[AMIT, ANIL, ANURAG]

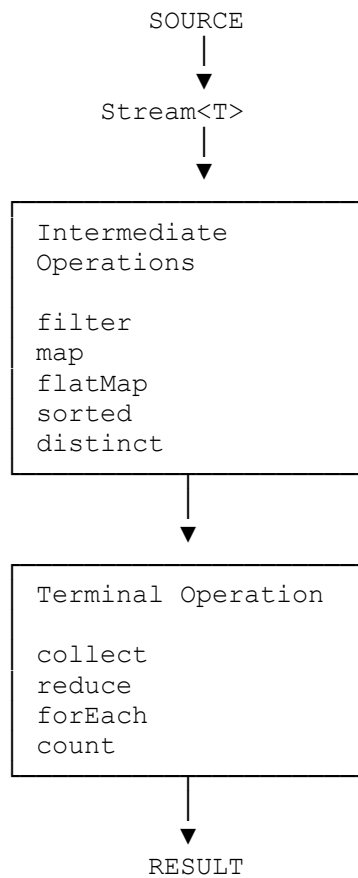
```

14. Core Streams API Architecture



15. Most Important Concept

The Streams API is fundamentally based on this pattern:



In one sentence

The Java 8 Streams API provides a functional, pipeline-based mechanism for processing sequences of data through lazy intermediate operations followed by a terminal operation that produces a result or side effect.

Java 8 — Optional

Java Version: 8

Year: 2014

Feature: Optional

UniqueClassName: Optional

1. Definition

Optional<T> is a **container object** introduced in Java 8 that may either:

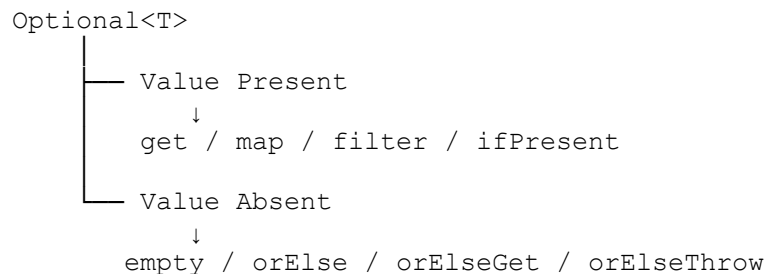
- contain a **non-null value**, or
- contain **no value** (empty).

It is primarily used to make the **absence of a value explicit** and reduce accidental NullPointerException caused by direct null handling.

Optional belongs to:

java.util.Optional

Basic idea



2. Why Optional Was Introduced

Before Java 8, code commonly handled potentially missing values like this:

```
String name = getName();

if (name != null) {
    System.out.println(name.toUpperCase());
} else {
    System.out.println("Name not available");
}
```

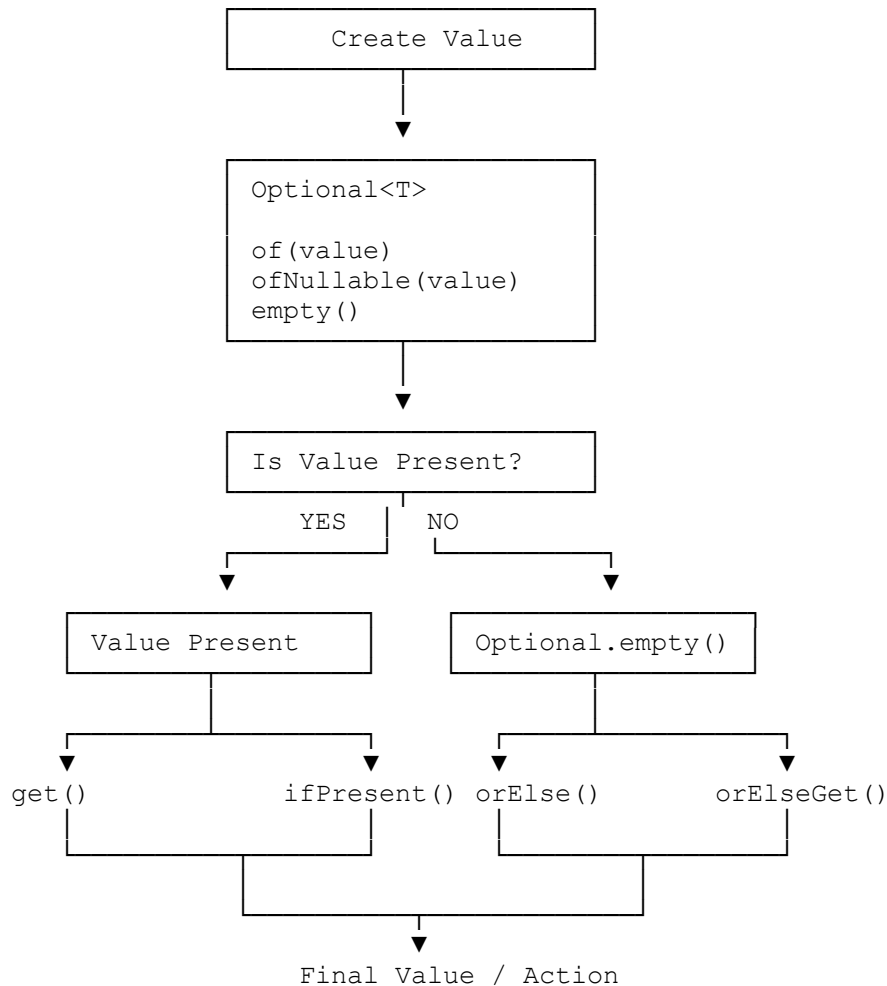
With Optional:

```
Optional<String> name = getName();
```

```
name.ifPresent(n -> System.out.println(n.toUpperCase()));
```

The important idea is that the API makes "**a value may be absent**" explicit in the type.

3. Optional Flowchart



4. Creating an Optional

There are three important ways to create an `Optional`.

`Optional.of()`

Use when the value is **definitely non-null**.

```
Optional<String> name = Optional.of("Anurag");
```

The value is:

```
"Anurag"
```

If you pass null:

```
Optional.of(null);
```

it throws:

```
NullPointerException
```

Optional.ofNullable()

Use when the value **may be null**.

```
String name = null;
```

```
Optional<String> optionalName =  
    Optional.ofNullable(name);
```

Result:

```
Optional.empty
```

If the value is non-null:

```
String name = "Anurag";
```

```
Optional<String> optionalName =  
    Optional.ofNullable(name);
```

Result:

```
Optional[Anurag]
```

Optional.empty()

Creates an empty Optional.

```
Optional<String> name = Optional.empty();
```

Conceptually:

Optional<String>



EMPTY

5. Complete Java Code Example

```
import java.util.Optional;

public class Optional {

    public static void main(String[] args) {

        String name = "Anurag";

        // Create Optional
        java.util.Optional<String> optionalName =
            java.util.Optional.ofNullable(name);

        // Check whether value exists
        if (optionalName.isPresent()) {
            System.out.println("Name = " + optionalName.get());
        } else {
            System.out.println("Name not available");
        }
    }
}
```

Output

Name = Anurag

Since the requested class name is Optional, it conflicts with java.util.Optional. Therefore, the example uses the fully qualified java.util.Optional type. In a real project, a class such as OptionalExample would avoid this naming conflict.

6. Better Practical Example

```
import java.util.Optional;

public class OptionalExample {

    public static void main(String[] args) {
```

```

        String name = null;

        Optional<String> optionalName =
            Optional.ofNullable(name);

        String result = optionalName
            .orElse("Name not available");

        System.out.println(result);
    }
}

```

Output

Name not available

Execution:

```

name = null
↓
Optional.ofNullable(name)
↓
Optional.empty()
↓
orElse("Name not available")
↓
"Name not available"

```

7. Important Optional Methods

Method	Purpose	Example
of()	Creates Optional from non-null value	Optional.of("Java")
ofNullable()	Creates Optional from nullable value	Optional.ofNullable(name)
empty()	Creates empty Optional	Optional.empty()
isPresent()	Checks whether value exists	optional.isPresent()
get()	Gets contained value	optional.get()
ifPresent()	Executes action when value exists	optional.ifPresent(System.out::println)

orElse()	Provides default value	optional.orElse("Unknown")
orElseGet()	Lazily generates default value	optional.orElseGet(() -> getDefault())
orElseThrow()	Throws exception if empty	optional.orElseThrow(...)
filter()	Keeps value if condition matches	optional.filter(x -> x > 10)
map()	Transforms contained value	optional.map(String::toUpperCase)
flatMap()	Maps to another Optional	optional.flatMap(...)

8. isPresent() and get()

Example:

```
Optional<String> name =
    Optional.of("Anurag");
```

```
if (name.isPresent()) {
    System.out.println(name.get());
}
```

Output:

Anurag

Flow:

```
Optional["Anurag"]
    ↓
isPresent()
    ↓
true
    ↓
get()
    ↓
"Anurag"
```

Important

Calling:

```
optional.get();
```

when the Optional is empty throws:

`NoSuchElementException`

Therefore, blindly using `get()` is generally not the best way to use `Optional`.

9. `ifPresent()`

Instead of:

```
if (optional.isPresent()) {  
    System.out.println(optional.get());  
}
```

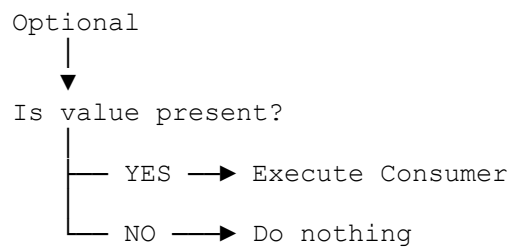
you can write:

```
optional.ifPresent(  
    value -> System.out.println(value)  
);
```

Or with a method reference:

```
optional.ifPresent(System.out::println);
```

Flow:



10. orElse()

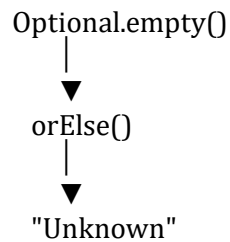
orElse() provides a fallback value.

```
Optional<String> name =  
    Optional.empty();  
  
String result =  
    name.orElse("Unknown");  
  
System.out.println(result);
```

Output:

Unknown

Flow:



If the value exists:

```
Optional<String> name =  
    Optional.of("Anurag");  
  
String result =  
    name.orElse("Unknown");
```

Result:

Anurag

11. orElseGet()

orElseGet() accepts a Supplier that generates the fallback value.

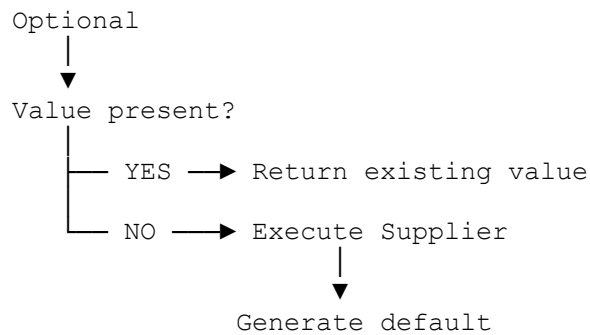
```
Optional<String> name =  
    Optional.empty();  
  
String result = name.orElseGet(  
    () -> "Default Name"
```

```
);  
  
System.out.println(result);
```

Output:

Default Name

Conceptually:



12. `orElse()` vs `orElseGet()`

This distinction is important.

```
optional.orElse(getDefault());
```

versus:

```
optional.orElseGet(() -> getDefault());
```

`orElse()` receives an already evaluated argument, whereas `orElseGet()` invokes the supplier only when the `Optional` is empty.

Example:

```
String result = optional.orElse(createDefault());
```

versus:

```
String result =  
    optional.orElseGet(() -> createDefault());
```

For expensive fallback computation, `orElseGet()` can avoid unnecessary work when the value is already present.

13. map()

map() transforms the value inside the Optional.

```
Optional<String> name =  
    Optional.of("anurag");  
  
Optional<String> upperName =  
    name.map(String::toUpperCase);  
  
System.out.println(upperName);
```

Output:

Optional[ANURAG]

Flow:

```
Optional["anurag"]  
    ↓  
  map()  
    ↓ String::toUpperCase  
Optional["ANURAG"]
```

If the original Optional is empty, map() returns an empty Optional.

14. filter()

filter() keeps the value only if the supplied condition is true.

```
Optional<Integer> number =  
    Optional.of(20);  
  
Optional<Integer> result =  
    number.filter(n -> n > 10);  
  
System.out.println(result);
```

Output:

Optional[20]

If:

```
Optional<Integer> number =  
    Optional.of(5);
```

then:

```
number.filter(n -> n > 10);
```

produces:

Optional.empty

Flow:

```
Optional[20]  
  ↓  
filter(n -> n > 10)  
  ↓  
20 > 10 → true  
  ↓  
Optional[20]
```

15. Optional with a Method Returning a Value

A common use is for methods whose result might not exist.

```
import java.util.Optional;  
  
public class OptionalExample {  
    public static Optional<String> findUserName(int id) {  
        if (id == 1) {  
            return Optional.of("Anurag");  
        }  
        return Optional.empty();  
    }  
  
    public static void main(String[] args) {  
        Optional<String> name =  
            findUserName(1);  
  
        System.out.println(  
            name.orElse("User not found")  
        );  
    }  
}
```

```
}  
}
```

Output:

Anurag

For another ID:

```
Optional<String> name =  
    findUserName(10);
```

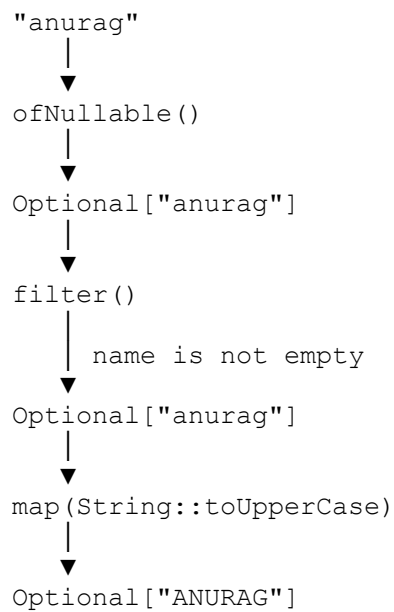
Output:

User not found

16. Complete Optional Processing Pipeline

```
Optional<String> result =  
    Optional.ofNullable("anurag")  
        .filter(name -> !name.isEmpty())  
        .map(String::toUpperCase);
```

Flow:



17. Optional vs null

Traditional null	Optional
Absence represented by null	Absence represented by <code>Optional.empty()</code>
Easy to forget null check	Absence is explicit in return type
Can cause <code>NullPointerException</code>	Provides APIs for handling absence
No standard transformation API	<code>map()</code> , <code>filter()</code> , <code>flatMap()</code>
Caller may not know value can be absent	<code>Optional<T></code> communicates possibility
Manual branching often required	Declarative handling possible

However, `Optional` does **not** make all `NullPointerExceptions` impossible. It is primarily a tool for explicitly modeling potentially absent values.

18. Important Design Principle

A common and useful use case is:

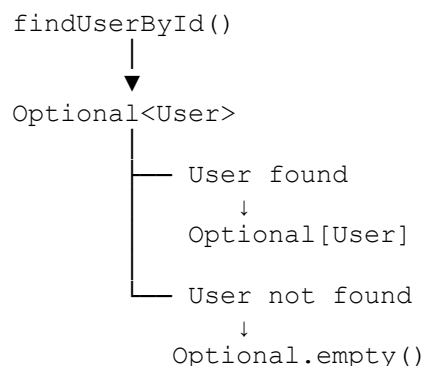
```
Optional<User> findUserById(int id)
```

instead of:

```
User findUserById(int id)
```

where null might secretly mean "user not found."

The first signature communicates:



19. Optional Is Not a Replacement for Every null

It is generally most useful for **return values where absence is a meaningful outcome**.

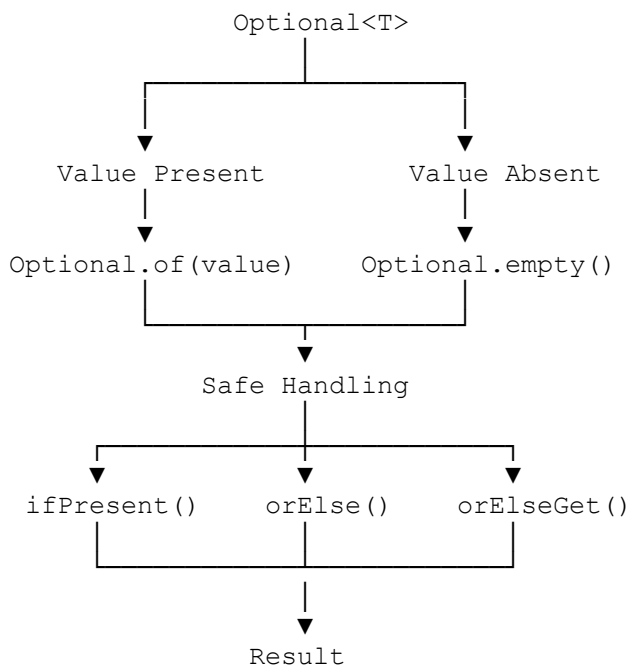
For example:

```
Optional<User> findUserById(int id);
```

is a natural use.

But using Optional indiscriminately for every field, parameter, or local variable is not automatically good design.

20. Key Concept



One-line definition

`Optional<T>` is a Java 8 container that represents either a non-null value or the explicit absence of a value, providing APIs such as `map()`, `filter()`, `ifPresent()`, `orElse()`, and `orElseGet()` for handling that state.

Java 8 — Method References

Java Version: 8

Year: 2014

Feature: Method References

UniqueClassName: MethodReferences

1. Definition

A **method reference** is a compact Java 8 syntax for referring to an **existing method without invoking it**.

It is an alternative to writing a lambda expression when the lambda's body simply calls an existing method.

Syntax

ClassName::methodName

or:

object::methodName

or:

ClassName::new

The :: operator is called the **method reference operator**.

Simple comparison

Lambda:

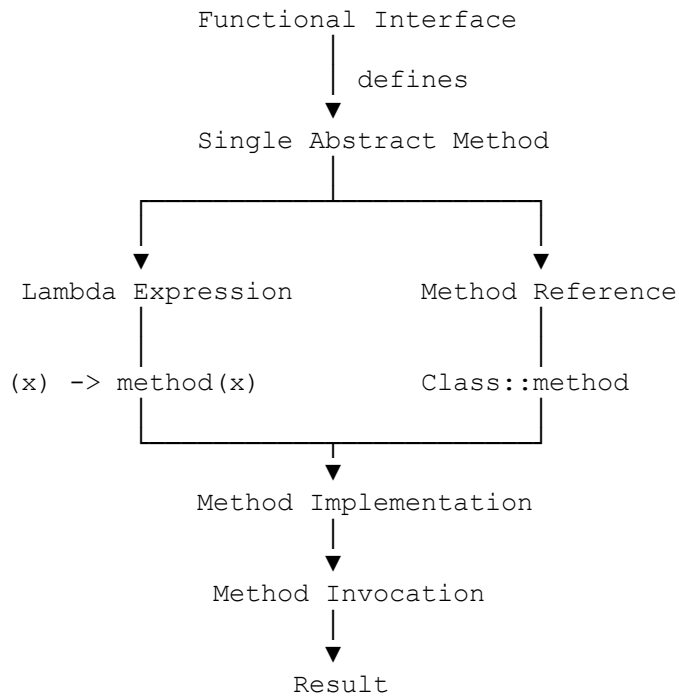
name -> System.out.println(name)

Method reference:

System.out::println

Both can represent the same behavior when the target functional interface has a compatible method signature.

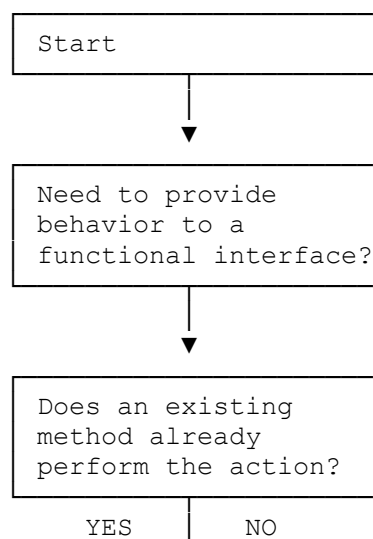
2. Relationship Between Functional Interface, Lambda, and Method Reference

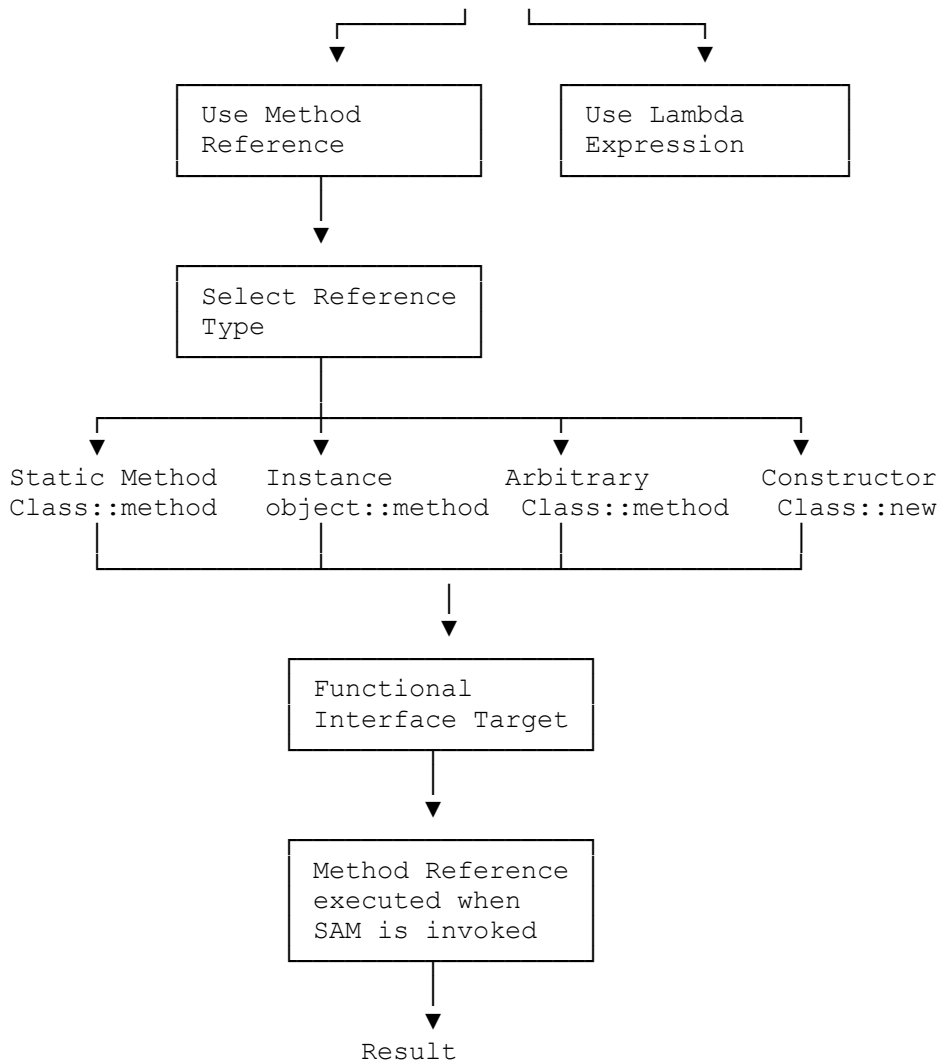


The key idea is:

A method reference is essentially a shorter way of expressing certain lambda expressions.

3. Method Reference Flowchart





4. Four Types of Method References

Java supports four major forms:

Type	Syntax	Example
1. Static method	ClassName::staticMethod	Math::abs
2. Instance method of a particular object	object::instanceMethod	System.out::println
3. Instance method of an arbitrary object of a type	ClassName::instanceMethod	String::toUpperCase
4. Constructor reference	ClassName::new	ArrayList::new

5. Type 1 — Static Method Reference

Syntax:

ClassName::staticMethod

Example:

Math::abs

Consider:

```
import java.util.function.Function;

public class MethodReferences {

    public static void main(String[] args) {

        Function<Integer, Integer> absoluteValue =
            Math::abs;

        System.out.println(absoluteValue.apply(-25));
    }
}
```

Output

25

Equivalent lambda

```
Function<Integer, Integer> absoluteValue =
    n -> Math.abs(n);
```

Method reference:

```
Function<Integer, Integer> absoluteValue =
    Math::abs;
```

The relationship is:

```
n -> Math.abs(n)
  |
  ▼
Math::abs
```

6. Type 2 — Instance Method of a Particular Object

Syntax:

`object::instanceMethod`

A classic example is:

`System.out::println`

Code:

```
import java.util.Arrays;
import java.util.List;

public class MethodReferences {

    public static void main(String[] args) {

        List<String> names =
            Arrays.asList("Anurag", "Amit", "Rahul");

        names.forEach(System.out::println);
    }
}
```

Output:

Anurag
Amit
Rahul

Equivalent lambda:

```
names.forEach(name -> System.out.println(name));
```

Method reference:

```
names.forEach(System.out::println);
```

Here:

System.out
|
▼
PrintStream object
|
▼
println()

The existing `println()` method is referenced rather than explicitly invoked in the method-reference expression.

7. Type 3 — Instance Method of an Arbitrary Object

Syntax:

`ClassName::instanceMethod`

This form can initially look confusing.

Example:

`String::toUpperCase`

Code:

```
import java.util.function.Function;

public class MethodReferences {

    public static void main(String[] args) {

        Function<String, String> converter =
            String::toUpperCase;

        System.out.println(
            converter.apply("java")
        );
    }
}
```

Output:

JAVA

Equivalent lambda:

```
Function<String, String> converter =
    text -> text.toUpperCase();
```

Method reference:

```
Function<String, String> converter =
    String::toUpperCase;
```

Conceptually:

Function.apply("java")
↓
String::toUpperCase
↓
"java".toUpperCase()
↓
"JAVA"

The object on which the instance method is called is supplied as the appropriate functional-interface argument.

8. Type 4 — Constructor Reference

Syntax:

ClassName::new

A constructor reference refers to a constructor instead of a method.

Example:

```
import java.util.ArrayList;
import java.util.function.Supplier;

public class MethodReferences {

    public static void main(String[] args) {

        Supplier<ArrayList<String>> creator =
            ArrayList::new;

        ArrayList<String> list = creator.get();

        list.add("Java");

        System.out.println(list);
    }
}
```

Output:

[Java]

Equivalent lambda:

```
Supplier<ArrayList<String>> creator =  
    () -> new ArrayList<>();
```

Method/constructor reference:

```
Supplier<ArrayList<String>> creator =  
    ArrayList::new;
```

9. Complete Java Code Example

The following program demonstrates all four types.

```
import java.util.ArrayList;  
import java.util.Arrays;  
import java.util.List;  
import java.util.function.Function;  
import java.util.function.Supplier;  
  
public class MethodReferences {  
  
    public static void main(String[] args) {  
  
        // 1. Static method reference  
        Function<Integer, Integer> absolute =  
            Math::abs;  
  
        System.out.println(  
            "Absolute = " + absolute.apply(-100)  
        );  
  
        // 2. Instance method reference  
        //    of a particular object  
        List<String> names =  
            Arrays.asList(  
                "Anurag",  
                "Amit",  
                "Rahul"  
            );  
  
        names.forEach(System.out::println);  
  
        // 3. Instance method reference  
        //    of an arbitrary object  
        Function<String, String> upperCase =  
            String::toUpperCase;  
  
        System.out.println(  
            "Uppercase = " + upperCase.apply("java")  
        );  
    }  
}
```

```

// 4. Constructor reference
Supplier<ArrayList<String>> creator =
    ArrayList::new;

ArrayList<String> list =
    creator.get();

list.add("Java");
list.add("Streams");

System.out.println(
    "List = " + list
);
    }
}

```

Output

```

Absolute = 100
Anurag
Amit
Rahul
Uppercase = JAVA
List = [Java, Streams]

```

10. Lambda vs Method Reference

Lambda Expression	Method Reference
n -> Math.abs(n)	Math::abs
name -> System.out.println(name)	System.out::println
s -> s.toUpperCase()	String::toUpperCase
() -> new ArrayList<>()	ArrayList::new

A method reference should generally be used when it makes the code **shorter and clearer**.

11. Method Reference with forEach()

Consider:

```
List<String> names =
```

```
Arrays.asList("A", "B", "C");
```

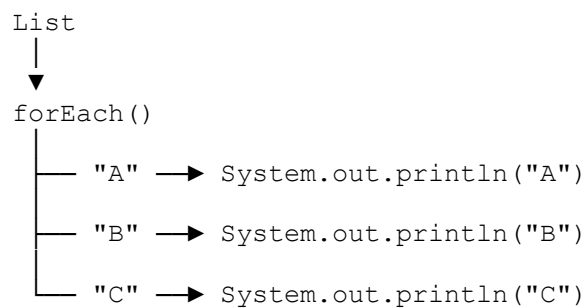
Using lambda:

```
names.forEach(  
    name -> System.out.println(name)  
);
```

Using method reference:

```
names.forEach(  
    System.out::println  
);
```

Flow:



12. Method Reference with Streams

Method references are particularly useful with the Java 8 Streams API.

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class MethodReferences {

    public static void main(String[] args) {

        List<String> names =
            Arrays.asList(
                "anurag",
                "amit",
                "rahul"
            );

        List<String> result =
            names.stream()
                .map(String::toUpperCase)
                .collect(Collectors.toList());
```

```
        System.out.println(result);
    }
}
```

Output:

[ANURAG, AMIT, RAHUL]

Equivalent lambda:

`.map(name -> name.toUpperCase())`

Method reference:

`.map(String::toUpperCase)`

13. Method Reference with Sorting

```
import java.util.Arrays;
import java.util.List;

public class MethodReferences {

    public static void main(String[] args) {

        List<String> names =
            Arrays.asList(
                "Vijay",
                "Amit",
                "Rahul",
                "Anurag"
            );

        names.sort(String::compareTo);

        System.out.println(names);
    }
}
```

Output:

[Amit, Anurag, Rahul, Vijay]

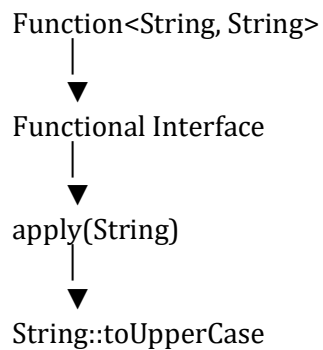
14. Method Reference and Functional Interface

A method reference needs a **target type**, commonly a functional interface.

For example:

```
Function<String, String> converter =  
    String::toUpperCase;
```

Here:



The compiler uses the target type to determine whether the referenced method is compatible with the functional interface's abstract method.

15. Important Difference: Method Reference Does Not Call the Method Immediately

This:

```
System.out::println
```

does **not** mean:

```
System.out.println();
```

It creates a method-reference expression that can be used where a compatible functional-interface target is expected.

For example:

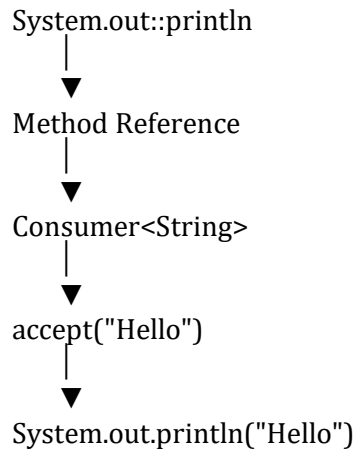
```
Consumer<String> printer =  
    System.out::println;
```

The actual invocation happens when:

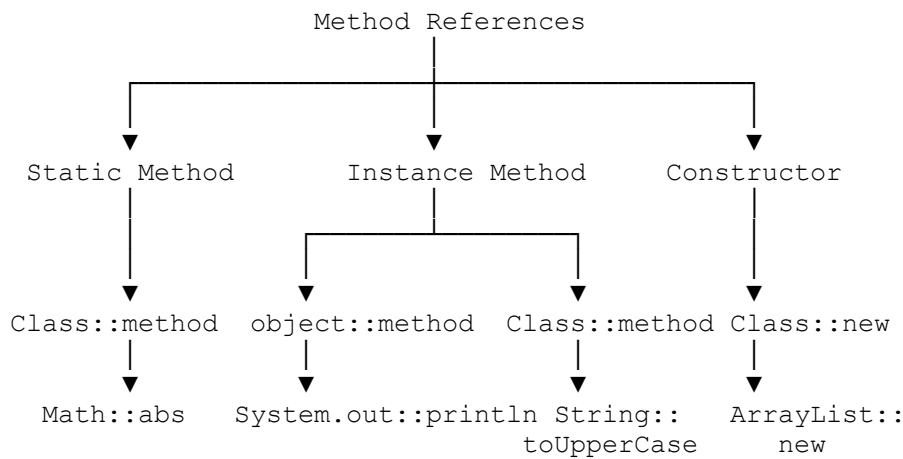
```
printer.accept("Hello");
```

is executed.

Flow:



16. Method Reference Types — Summary



17. Key Rules

1. Method references were introduced in **Java 8**.
2. They use the `::` operator.
3. They refer to an existing method or constructor.
4. They do **not** invoke the method merely by being written.
5. They are commonly used with functional interfaces.
6. They are an alternative to certain lambda expressions.
7. There are four primary forms:
 - `ClassName::staticMethod`
 - `object::instanceMethod`

- ClassName::instanceMethod
 - ClassName::new
8. They are heavily used with the **Streams API**, collections, and functional interfaces.
 9. The referenced method must be compatible with the target functional interface's abstract method.

One-line definition

A method reference in Java 8 is a compact `::` syntax that refers to an existing method or constructor and can be used as the implementation of a compatible functional interface, often replacing a lambda expression that merely delegates to that method.

Java 8 — Default Interface Methods

Java Version: 8

Year: 2014

Feature: Default Interface Methods

UniqueClassName: DefaultMethods

1. Definition

A **default interface method** is a method inside an interface that has a **concrete implementation** and is declared using the `default` keyword.

Default methods were introduced in **Java 8** primarily to allow interfaces to evolve by adding new behavior **without forcing every existing implementing class to provide an implementation**.

Syntax

```
interface InterfaceName {  
  
    default void methodName() {  
        // implementation  
    }  
}
```

For example:

```
interface Vehicle {  
  
    default void start() {  
        System.out.println("Vehicle is starting");  
    }  
}
```

```
}
```

A class implementing Vehicle automatically inherits start():

```
class Car implements Vehicle {  
}
```

Therefore:

```
Car car = new Car();  
car.start();
```

produces:

Vehicle is starting

2. Why Default Methods Were Introduced

Before Java 8, an interface method was traditionally abstract:

```
interface Vehicle {  
    void start();  
}
```

If thousands of classes implemented Vehicle and Java added:

```
void stop();
```

every implementing class would need to implement the new method.

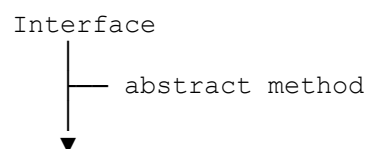
Java 8 solves this problem with:

```
default void stop() {  
    System.out.println("Vehicle stopped");  
}
```

Existing classes can continue working without immediately implementing the new method.

Concept

Before Java 8



Every implementing class
must implement it

Java 8

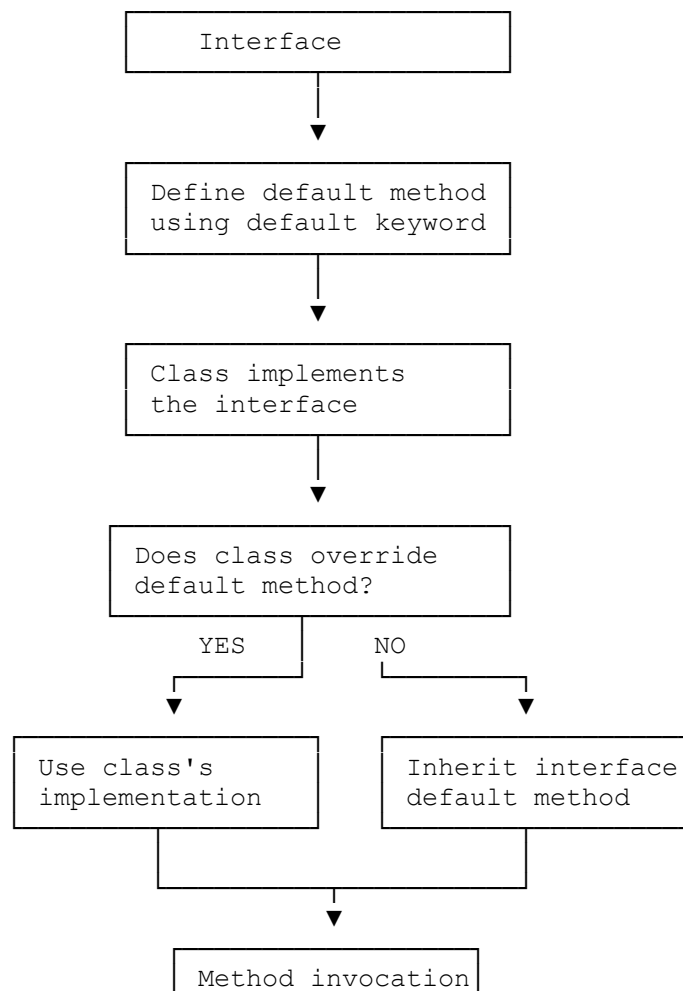
Interface

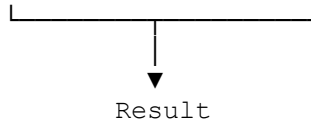
- abstract method
- default method

Provides implementation

Implementing class may
inherit or override it

3. Default Method Flowchart





4. Basic Java Code Example

```
interface Vehicle {  
    default void start() {  
        System.out.println("Vehicle is starting");  
    }  
}  
  
class Car implements Vehicle {  
    // No need to implement start()  
}  
  
public class DefaultMethods {  
    public static void main(String[] args) {  
        Car car = new Car();  
        car.start();  
    }  
}
```

Output

Vehicle is starting

Here:

default void start()

has a complete implementation inside the interface.

Car inherits this implementation.

5. Complete Execution Flow

```
Vehicle interface  
  |  
  ▼  
default start()  
  |
```

```
    | implementation:
    | System.out.println(...)
    ▼
Car implements Vehicle
    |
    ▼
Car does not override start()
    |
    ▼
Car inherits start()
    |
    ▼
car.start()
    |
    ▼
"Vehicle is starting"
```

6. Overriding a Default Method

A class is allowed to override a default method.

```
interface Vehicle {

    default void start() {
        System.out.println("Vehicle is starting");
    }
}

class Car implements Vehicle {

    @Override
    public void start() {
        System.out.println("Car is starting");
    }
}

public class DefaultMethods {

    public static void main(String[] args) {

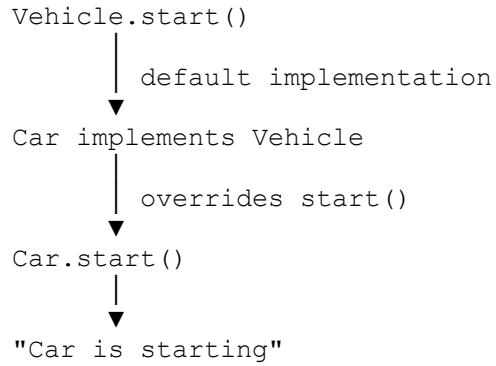
        Car car = new Car();

        car.start();
    }
}
```

Output

Car is starting

Flow:



The class's overridden method takes precedence over the inherited default method.

7. Calling the Interface's Default Implementation

Java provides:

`InterfaceName.super.methodName();`

to explicitly invoke the default method from the implementing class.

Example:

```
interface Vehicle {
    default void start() {
        System.out.println("Vehicle is starting");
    }
}

class Car implements Vehicle {
    @Override
    public void start() {
        System.out.println("Car is starting");
        Vehicle.super.start();
    }
}

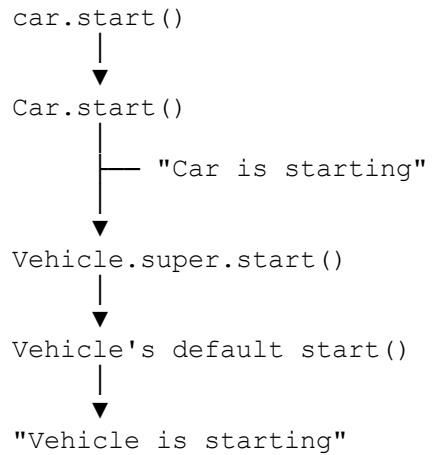
public class DefaultMethods {
    public static void main(String[] args) {
        Car car = new Car();
        car.start();
    }
}
```

```
}
```

Output

Car is starting
Vehicle is starting

Flow:



8. Multiple Interfaces with Default Methods

A class can implement multiple interfaces.

```
interface A {
    default void show() {
        System.out.println("A");
    }
}

interface B {
    default void show() {
        System.out.println("B");
    }
}

class Test implements A, B {
    @Override
    public void show() {
        System.out.println("Test");
    }
}
```

Why is overriding necessary?

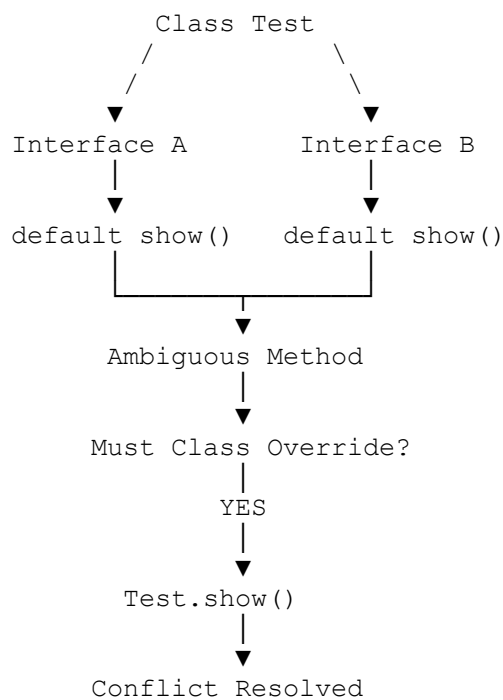
Because both interfaces provide:

default void show()

Java cannot automatically decide which implementation to inherit.

Therefore, the class must resolve the conflict.

9. Default Method Conflict Flowchart



Example:

```
interface A {
    default void show() {
        System.out.println("A");
    }
}

interface B {
    default void show() {
        System.out.println("B");
    }
}
```

```

}

class Test implements A, B {

    @Override
    public void show() {

        System.out.println("Test");

        A.super.show();
        B.super.show();
    }
}

```

Output:

```

Test
A
B

```

The class can explicitly choose either interface's default implementation.

10. Interface Inheritance and Default Methods

Consider:

```

interface Parent {

    default void display() {
        System.out.println("Parent");
    }
}

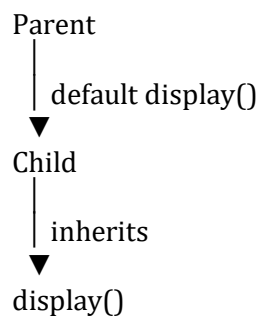
```

```

interface Child extends Parent {
}

```

Child inherits the default method from Parent.



Example:

```
class Test implements Child {  
}  
  
public class DefaultMethods {  
    public static void main(String[] args) {  
        Test test = new Test();  
        test.display();  
    }  
}
```

Output:

Parent

11. Child Interface Can Override Default Method

```
interface Parent {  
    default void display() {  
        System.out.println("Parent");  
    }  
}  
  
interface Child extends Parent {  
    @Override  
    default void display() {  
        System.out.println("Child");  
    }  
}  
  
class Test implements Child {  
}
```

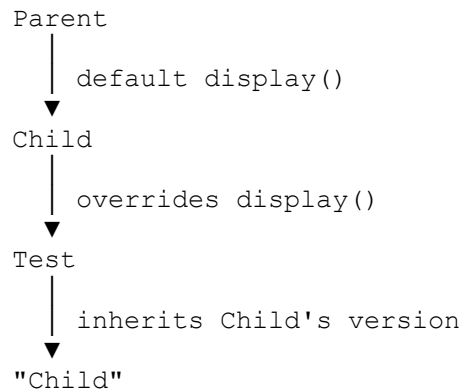
Now:

```
Test test = new Test();  
test.display();
```

Output:

Child

Hierarchy:



12. Default Methods and Functional Interfaces

A functional interface can contain default methods.

For example:

```
@FunctionalInterface
interface Calculator {

    int calculate(int a, int b);

    default void message() {
        System.out.println("Calculator");
    }
}
```

This is still a functional interface because:

calculate() → abstract
message() → default

Abstract methods = 1

Therefore, a lambda can implement it:

Calculator addition =
(a, b) -> a + b;

Complete example:

```
@FunctionalInterface
interface Calculator {
```

```

        int calculate(int a, int b);

        default void message() {
            System.out.println("Calculator is ready");
        }
    }

    public class DefaultMethods {

        public static void main(String[] args) {

            Calculator calculator =
                (a, b) -> a + b;

            calculator.message();

            System.out.println(
                calculator.calculate(10, 20)
            );
        }
    }
}

```

Output:

Calculator is ready
30

13. Default Method vs Abstract Method

Feature	Abstract Interface Method	Default Interface Method
Implementation	No	Yes
Keyword	No default	default
Implementing class must implement?	Normally yes	No
Can be overridden?	Must be implemented	Yes
Can call Interface.super.method()?	No	Yes
Java version	Traditional	Java 8
Main purpose	Define contract	Add reusable behavior

Example:

Abstract

```
interface Vehicle {  
  
    void start();  
}
```

The class must provide:

```
class Car implements Vehicle {  
  
    @Override  
    public void start() {  
        System.out.println("Car");  
    }  
}
```

Default

```
interface Vehicle {  
  
    default void start() {  
        System.out.println("Vehicle");  
    }  
}
```

The class can simply:

```
class Car implements Vehicle {  
}
```

14. Default Method vs Static Interface Method

Java 8 also introduced static methods in interfaces.

```
interface Vehicle {  
  
    default void start() {  
        System.out.println("Starting");  
    }  
  
    static void info() {  
        System.out.println("Vehicle information");  
    }  
}
```

The default method is invoked through an object:

```
Car car = new Car();  
car.start();
```

The static method is invoked through the interface:

```
Vehicle.info();
```

Default Method	Static Interface Method
Uses default	Uses static
Inherited by implementing classes	Not inherited as an instance method
Can be overridden	Cannot be overridden as an interface static method
Called through implementing object	Called using interface name
<code>car.start()</code>	<code>Vehicle.info()</code>

15. Real-World Reason for Default Methods

One major motivation was **interface evolution**.

Suppose Java originally had:

```
interface Payment {  
  
    void pay();  
}
```

Many classes implement it:

```
Payment  
├── CreditCardPayment  
├── CashPayment  
├── BankPayment  
└── MobilePayment
```

Later, a new common operation is needed:

```
void receipt();
```

If it is abstract:

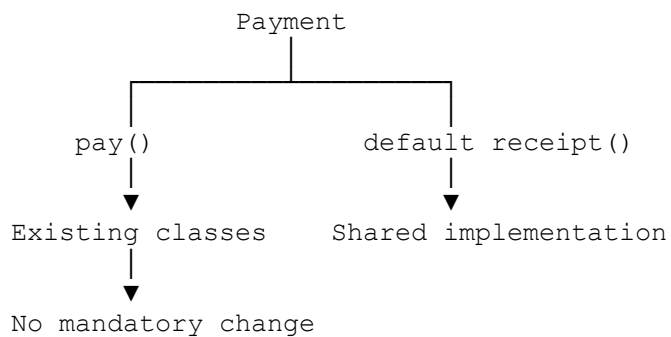
```
interface Payment {  
  
    void pay();  
  
    void receipt();  
}
```

every implementation must change.

Instead:

```
interface Payment {  
  
    void pay();  
  
    default void receipt() {  
        System.out.println("Generating receipt");  
    }  
}
```

Existing implementations can continue working.



16. Important Rules

Rule 1 — Use default inside an interface

```
interface A {  
  
    default void test() {  
        System.out.println("Test");  
    }  
}
```

Rule 2 — Default methods must have a body

Invalid:

```
interface A {  
    default void test();  
}
```

Valid:

```
interface A {  
    default void test() {  
        System.out.println("Test");  
    }  
}
```

Rule 3 — Implementing classes can override them

```
class B implements A {  
    @Override  
    public void test() {  
        System.out.println("B");  
    }  
}
```

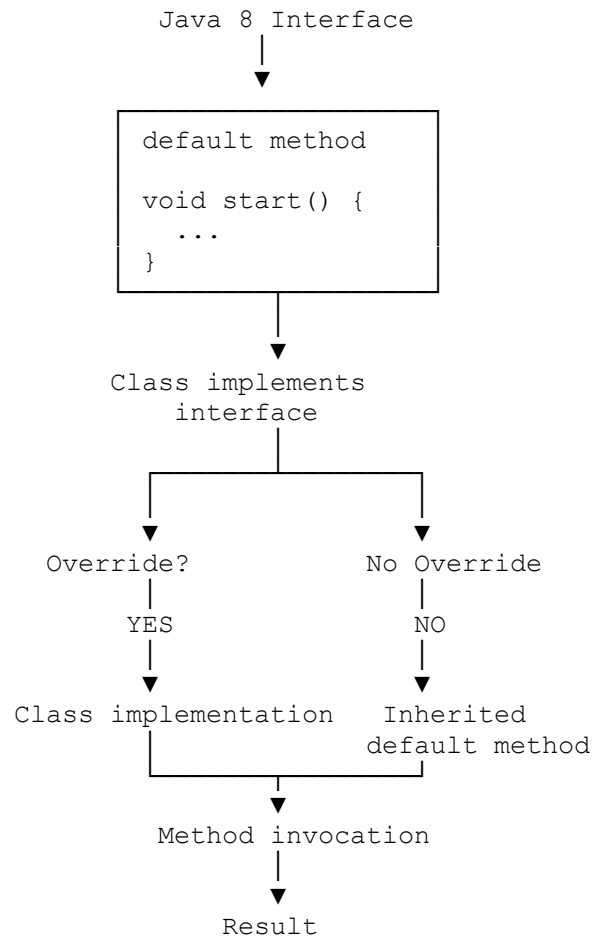
Rule 4 — Multiple default-method conflicts must be resolved

```
A.show()  
B.show()  
↓  
Conflict  
↓  
Implementing class must override show()
```

Rule 5 — Interface default implementation can be explicitly invoked

```
A.super.show();
```

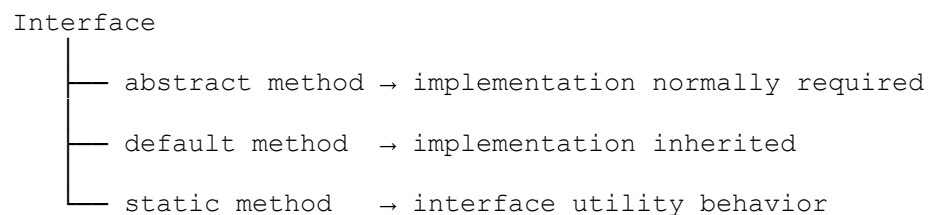
17. Complete Conceptual Flow



18. Key Concept

A Java 8 default interface method is a concrete method declared with the default keyword inside an interface, allowing implementing classes to inherit a ready-made implementation while still retaining the ability to override it.

The central idea is:



Default methods were therefore a major Java 8 feature for **interface evolution, backward compatibility, reusable behavior, and APIs such as the Collections framework.**

Java 8 — java.time Date-Time API

Java Version: 8

Year: 2014

Feature: java.time Date-Time API

UniqueClassName: DateTimeAPI

1. Definition

The **java.time Date-Time API** was introduced in **Java 8** through **JSR-310** to provide a modern, clear, immutable, and thread-safe API for handling dates, times, durations, periods, time zones, and date-time formatting.

It was designed to address many limitations of the older:

java.util.Date
java.util.Calendar

APIs.

The main package is:

java.time

Important related packages include:

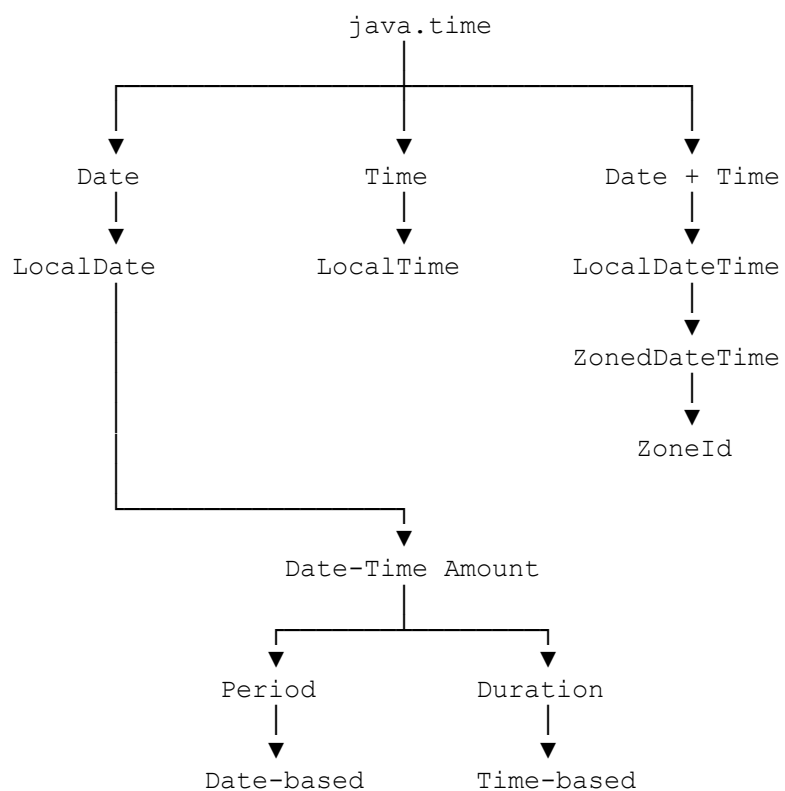
java.time
java.time.format
java.time.temporal
java.time.zone

2. Main java.time Classes

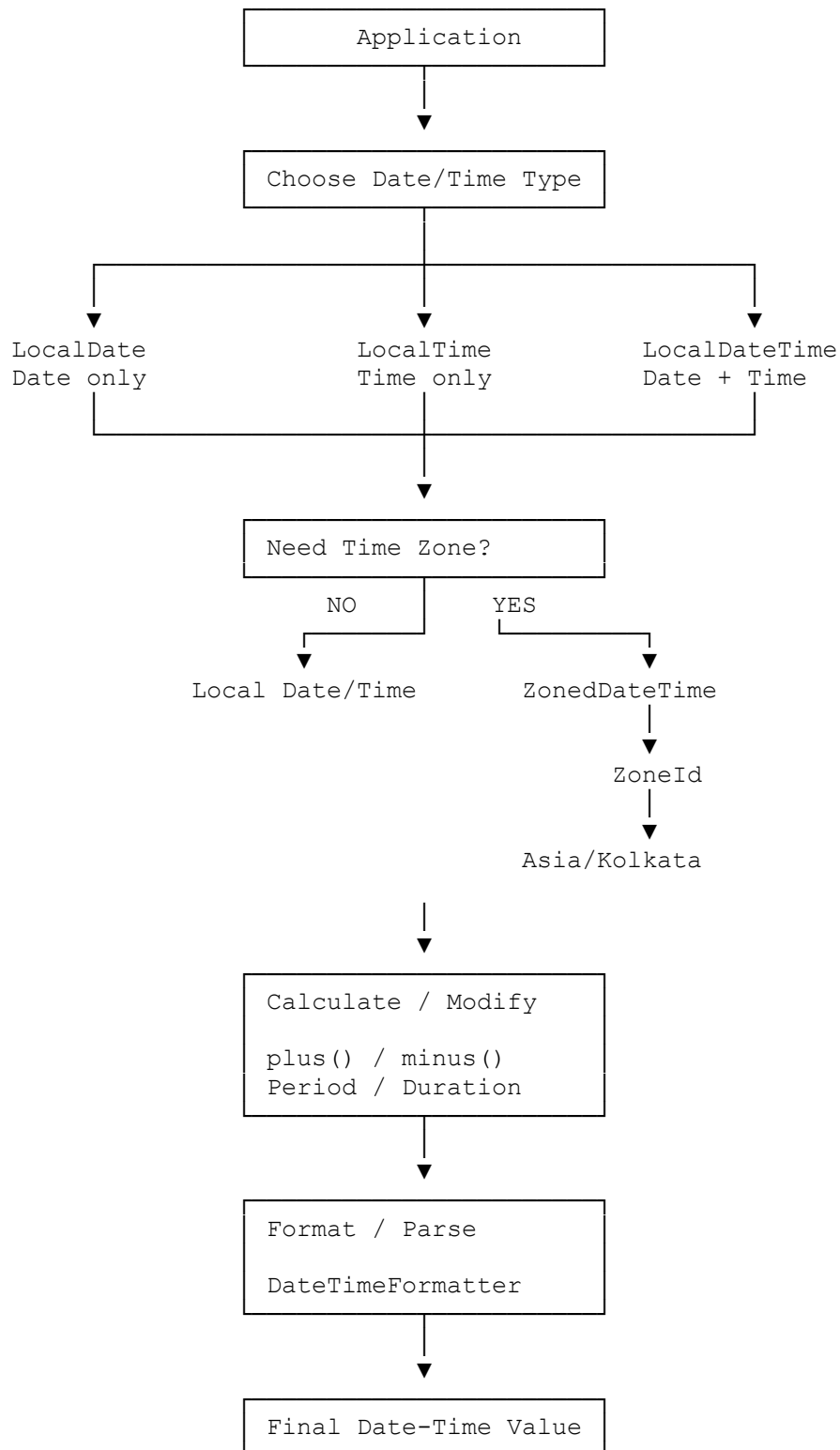
Class	Represents	Example
LocalDate	Date without time/time zone	2026-08-25
LocalTime	Time without date/time zone	11:30:45
LocalDateTime	Date + time without time zone	2026-08-25T11:30:45
ZonedDateTime	Date + time + time zone	2026-08-25T11:30+05:30[Asia/Kolkata]

OffsetDateTime	Date + time + UTC offset	2026-08-25T11:30+05:30
Instant	Point on the UTC timeline	2026-08-25T06:00:00Z
Duration	Time-based amount	2 hours
Period	Date-based amount	2 years, 3 months
ZoneId	Time-zone identifier	Asia/Kolkata
ZoneOffset	UTC offset	+05:30
DateTimeFormatter	Formatting/parsing	25-08-2026

3. java.time Architecture



4. Date-Time API Flowchart



5. Complete Java Code Example

```
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.time.Period;
import java.time.ZoneId;
import java.time.ZonedDateTime;
import java.time.format.DateTimeFormatter;

public class DateTimeAPI {

    public static void main(String[] args) {

        // Date
        LocalDate date = LocalDate.of(2026, 8, 25);

        // Time
        LocalTime time = LocalTime.of(11, 30, 45);

        // Date + Time
        LocalDateTime dateTime =
            LocalDateTime.of(date, time);

        // Time zone
        ZoneId zone = ZoneId.of("Asia/Kolkata");

        // Date + Time + Zone
        ZonedDateTime zonedDateTime =
            ZonedDateTime.of(dateTime, zone);

        // Add a period
        LocalDate futureDate =
            date.plus(Period.ofDays(10));

        // Format
        DateTimeFormatter formatter =
            DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm:ss");

        String formatted =
            dateTime.format(formatter);

        System.out.println("Date          = " + date);
        System.out.println("Time          = " + time);
        System.out.println("DateTime       = " + dateTime);
        System.out.println("ZonedDateTime = " + zonedDateTime);
        System.out.println("Future Date    = " + futureDate);
        System.out.println("Formatted      = " + formatted);
    }
}
```

Output

```
Date          = 2026-08-25
Time          = 11:30:45
DateTime       = 2026-08-25T11:30:45
```

```
ZonedDateTime = 2026-08-25T11:30:45+05:30[Asia/Kolkata]
Future Date    = 2026-09-04
Formatted      = 25-08-2026 11:30:45
```

6. LocalDate

LocalDate represents a **date without a time and without a time zone**.

```
LocalDate date =
    LocalDate.of(2026, 8, 25);
```

```
System.out.println(date);
```

Output:

2026-08-25

Common operations:

```
LocalDate date = LocalDate.of(2026, 8, 25);

System.out.println(date.getYear());
System.out.println(date.getMonth());
System.out.println(date.getDayOfMonth());

System.out.println(date.plusDays(10));
System.out.println(date.minusMonths(2));
```

Conceptually:

```
LocalDate
|
├── Year
├── Month
└── Day
```

7. LocalTime

LocalTime represents a **time without a date and without a time zone**.

```
LocalTime time =
    LocalTime.of(11, 30, 45);
```

```
System.out.println(time);
```

Output:

11:30:45

It can be modified:

```
LocalTime newTime =  
    time.plusHours(2)  
        .plusMinutes(15);
```

Conceptually:

```
LocalTime  
|  
├── Hour  
├── Minute  
├── Second  
└── Nanosecond
```

8. LocalDateTime

LocalDateTime combines:

LocalDate + LocalTime

Example:

```
LocalDateTime dateTime =  
    LocalDateTime.of(  
        2026,  
        8,  
        25,  
        11,  
        30  
    );
```

```
System.out.println(dateTime);
```

Output:

2026-08-25T11:30

Flow:

```
LocalDate  
2026-08-25  
└──┬──  
    |
```

```
LocalTime
11:30
    |
    v
LocalDateTime
2026-08-25T11:30
```

9. ZonedDateTime

When the application needs a **time zone**, use `ZonedDateTime`.

```
ZoneId zone =
    ZoneId.of("Asia/Kolkata");

ZonedDateTime dateTime =
    ZonedDateTime.now(zone);

System.out.println(dateTime);
```

A value contains:

```
Date
+
Time
+
Time Zone
+
Offset
```

Conceptually:

```
ZonedDateTime
|
├── LocalDate
├── LocalTime
├── ZoneId
└── ZoneOffset
```

10. ZoneId

`ZoneId` represents a geographical time-zone identifier.

Examples:

```
ZoneId.of("Asia/Kolkata");
ZoneId.of("America/New_York");
ZoneId.of("Europe/London");
```

Example:

```
ZoneId zone =  
    ZoneId.of("Asia/Kolkata");  
  
System.out.println(zone);
```

Output:

Asia/Kolkata

The important distinction is:

```
ZoneId  
↓  
Identifies a region/time-zone rule  
  
ZoneOffset  
↓  
Represents a specific offset from UTC
```

11. Instant

Instant represents a specific point on the **UTC timeline**.

```
import java.time.Instant;  
  
Instant instant = Instant.now();  
  
System.out.println(instant);
```

Example output:

2026-08-25T06:00:00Z

The Z indicates UTC.

Conceptually:

```
Local Date-Time  
|  
▼  
Time Zone / Offset  
|  
▼  
Instant  
|
```



UTC Timeline

Instant is especially useful for timestamps, logging, event ordering, and machine-to-machine communication.

12. Period

Period represents a **date-based amount**, such as:

Years
Months
Days

Example:

```
LocalDate date =  
    LocalDate.of(2026, 8, 25);
```

```
Period period =  
    Period.ofMonths(2);
```

```
LocalDate result =  
    date.plus(period);
```

```
System.out.println(result);
```

Output:

2026-10-25

Flow:

```
2026-08-25  
    |  
    + Period.ofMonths(2)  
    ▼  
2026-10-25
```

13. Duration

Duration represents a **time-based amount**.

For example:

```
import java.time.Duration;
import java.time.LocalDateTime;

LocalTime start =
    LocalDateTime.of(10, 0);

LocalTime end =
    LocalDateTime.of(12, 30);

Duration duration =
    Duration.between(start, end);

System.out.println(duration);
```

Output:

PT2H30M

Meaning:

2 hours 30 minutes

Difference

Period	Duration
Date-based	Time-based
Years	Hours
Months	Minutes
Days	Seconds
Used with dates	Used with times

14. Date-Time Formatting

Java 8 provides `DateTimeFormatter`.

```
LocalDate date =
    LocalDate.of(2026, 8, 25);
```

```
DateTimeFormatter formatter =
    DateTimeFormatter.ofPattern("dd/MM/yyyy");
```

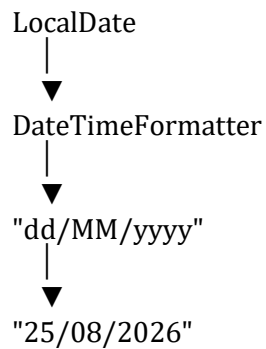
```
String result =  
    date.format(formatter);
```

```
System.out.println(result);
```

Output:

25/08/2026

Flow:



15. Parsing a String into a Date

The API can also convert text into date-time objects.

```
String text = "25-08-2026";
```

```
DateTimeFormatter formatter =  
    DateTimeFormatter.ofPattern("dd-MM-yyyy");
```

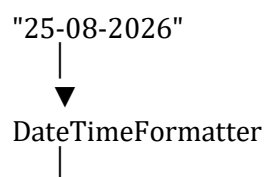
```
LocalDate date =  
    LocalDate.parse(text, formatter);
```

```
System.out.println(date);
```

Output:

2026-08-25

Flow:



▼
LocalDate.parse()
|
▼
2026-08-25

16. Immutability

The java.time classes are generally **immutable**.

For example:

```
LocalDate date =  
    LocalDate.of(2026, 8, 25);
```

```
LocalDate newDate =  
    date.plusDays(10);
```

```
System.out.println(date);  
System.out.println(newDate);
```

Output:

```
2026-08-25  
2026-09-04
```

The original date was not changed.

Conceptually:

```
date  
|  
| plusDays(10)  
▼  
newDate
```

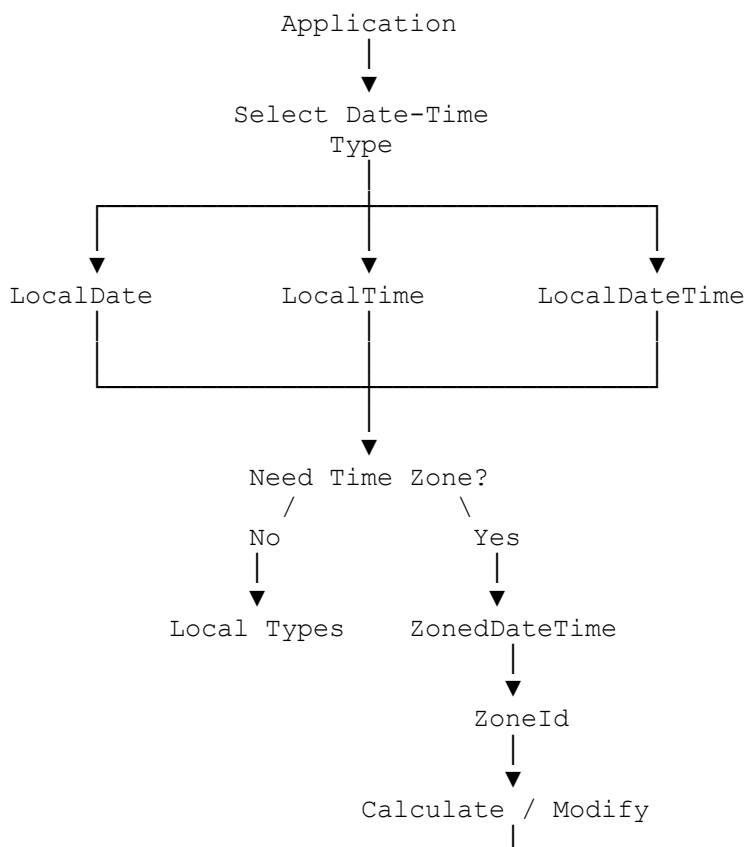
Original date remains unchanged

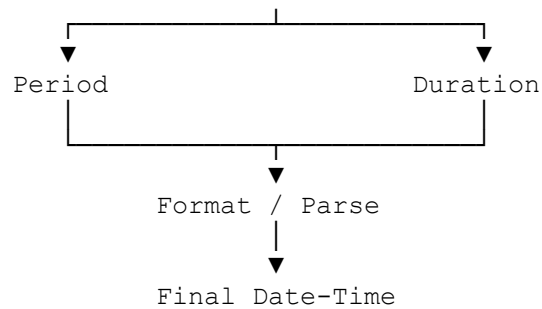
This makes the API easier to reason about and safer for concurrent use.

17. Old API vs Java 8 java.time

Old API	Java 8 API
<code>java.util.Date</code>	Instant, LocalDate, etc.
<code>java.util.Calendar</code>	LocalDate, ZonedDateTime, etc.
<code>SimpleDateFormat</code>	<code>DateTimeFormatter</code>
Mutable date/time objects	Immutable date/time objects
More error-prone API	Cleaner type-specific API
Calendar month indexing had historical quirks	Month enum / clearer API
Thread-safety concerns with <code>SimpleDateFormat</code>	<code>DateTimeFormatter</code> is immutable and thread-safe

18. Complete Execution Flow





19. Important Characteristics

1. Immutable

Operations produce new objects instead of changing existing ones.

2. Thread-safe

The core date-time value types are designed to be safely shared because they are immutable.

3. Type-specific

Instead of one general-purpose class, Java provides different types for different concepts:

Date only	→ LocalDate
Time only	→ LocalTime
Date + time	→ LocalDateTime
Date + time + zone	→ ZonedDateTime
UTC point	→ Instant
Date amount	→ Period
Time amount	→ Duration

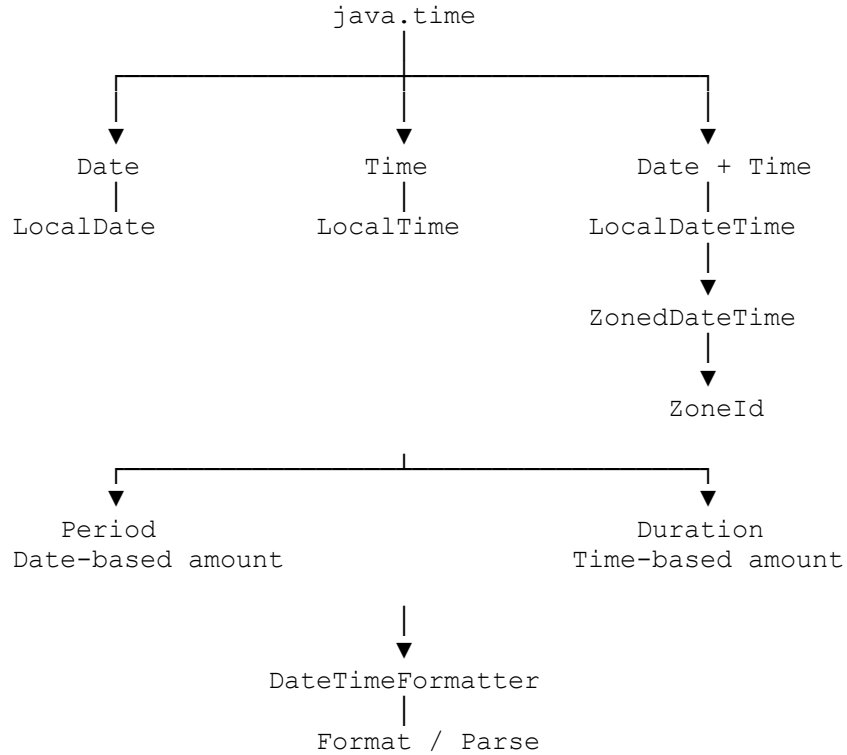
4. Clear API

Methods such as:

```
plusDays()  
minusMonths()  
plusHours()  
isBefore()  
isAfter()
```

make the intent explicit.

20. Key Concept



One-line definition

The Java 8 `java.time` Date-Time API is a modern, immutable, and thread-safe date/time framework that provides type-specific classes for dates, times, time zones, UTC instants, durations, periods, and formatting/parsing.

Java 8 — CompletableFuture

Java Version: 8

Year: 2014

Feature: CompletableFuture

UniqueClassName: CompletableFuture

1. Definition

CompletableFuture<T> is a Java 8 class in java.util.concurrent used for **asynchronous, non-blocking-style programming** and for composing multiple asynchronous operations.

It implements:

Future<T>

and also:

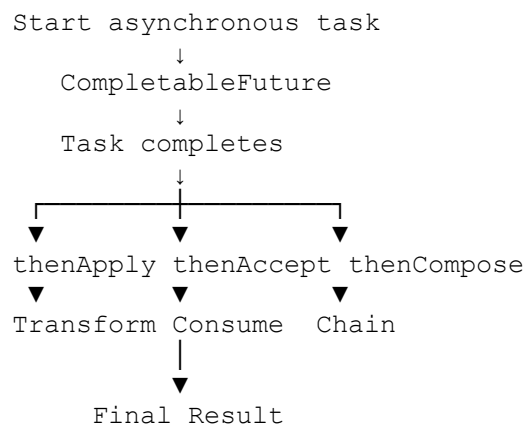
CompletionStage<T>

Unlike a traditional Future, a CompletableFuture allows you to **explicitly complete a computation**, attach dependent actions, combine asynchronous tasks, handle exceptions, and build asynchronous processing pipelines.

Package

java.util.concurrent.CompletableFuture

Core idea



2. Why CompletableFuture Was Introduced

Before Java 8, asynchronous programming commonly used Future.

Example:

```
Future<Integer> future =  
    executor.submit(() -> calculate());
```

To obtain the result:

```
Integer result = future.get();
```

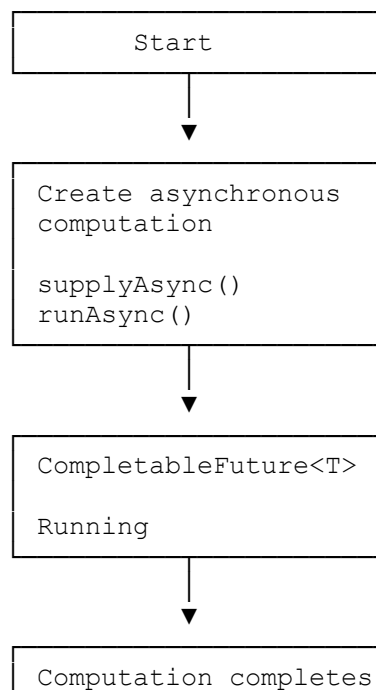
The problem is that `get()` can block the current thread.

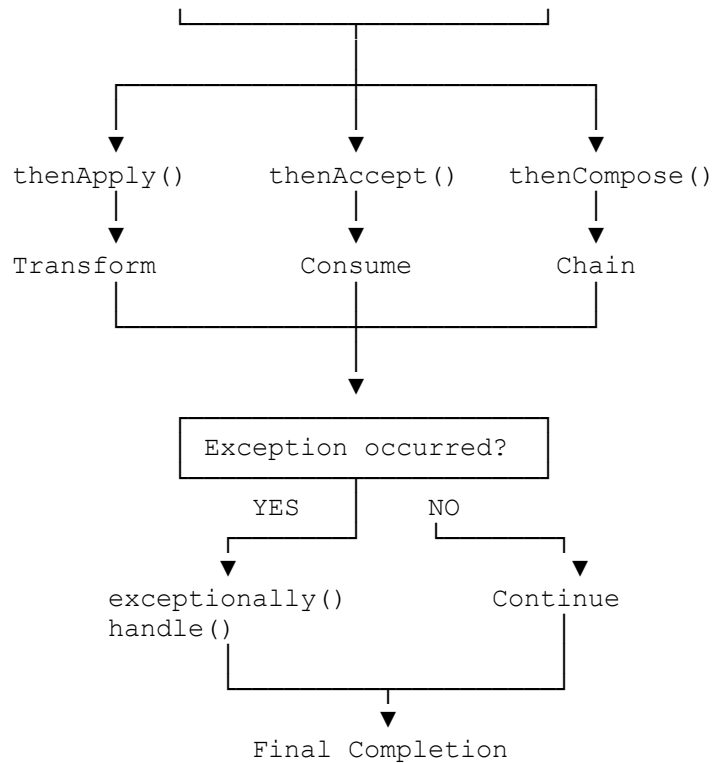
`CompletableFuture` allows you to express dependent work:

```
CompletableFuture  
    .supplyAsync(() -> calculate())  
    .thenApply(result -> result * 2)  
    .thenAccept(System.out::println);
```

Instead of manually waiting for each step, you can build an **asynchronous pipeline**.

3. CompletableFuture Flowchart





4. Basic CompletableFuture Example

```
import java.util.concurrent.CompletableFuture;

public class CompletableFuture {

    public static void main(String[] args) {

        java.util.concurrent.CompletableFuture<String> future =
            java.util.concurrent.CompletableFuture.supplyAsync(
                () -> "Hello Java"
            );

        String result = future.join();

        System.out.println(result);
    }
}
```

Output

Hello Java

Again, because the requested class is named `CompletableFuture`, the example uses the fully qualified Java type to avoid a naming conflict.

5. Recommended Example Class Name

In a real project, it is better to use:

```
public class CompletableFutureExample
```

rather than:

```
public class CompletableFuture
```

because the latter has the same simple name as:

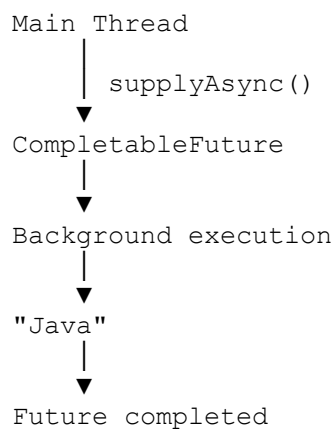
```
java.util.concurrent.CompletableFuture
```

6. supplyAsync()

`supplyAsync()` is used when an asynchronous task **returns a value**.

```
CompletableFuture<String> future =  
    CompletableFuture.supplyAsync(  
        () -> "Java"  
    );
```

Conceptually:



7. runAsync()

runAsync() is used when an asynchronous task **does not return a value**.

```
CompletableFuture<Void> future =  
    CompletableFuture.runAsync(  
        () -> System.out.println("Running asynchronously")  
    );
```

The result type is:

CompletableFuture<Void>

because there is no returned value.

Difference

Method	Returns value?	Result
runAsync()	No	CompletableFuture<Void>
supplyAsync()	Yes	CompletableFuture<T>

8. thenApply()

thenApply() transforms the result of a completed stage.

Example:

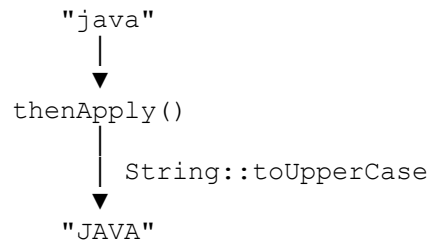
```
CompletableFuture<String> future =  
    CompletableFuture.  
        .supplyAsync(() -> "java")  
        .thenApply(String::toUpperCase);  
  
System.out.println(future.join());
```

Output:

JAVA

Flow:

```
supplyAsync()  
  ↓  
  ▼
```



Lambda version

```
.thenApply(value -> value.toUpperCase())
```

Method reference version

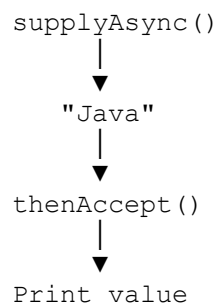
```
.thenApply(String::toUpperCase)
```

9. thenAccept()

thenAccept() consumes the result but does not produce another value.

```
CompletableFuture
    .supplyAsync(() -> "Java")
    .thenAccept(
        value -> System.out.println(value)
    );
```

Flow:



The resulting type is:

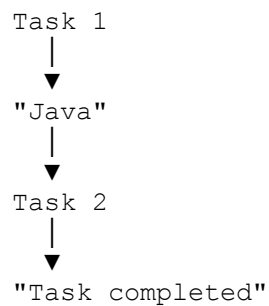
```
CompletableFuture<Void>
```

10. thenRun()

`thenRun()` executes an action after the previous stage completes, but it does not receive the previous result.

```
CompletableFuture
    .supplyAsync() -> "Java")
    .thenRun() ->
        System.out.println("Task completed")
    );
```

Conceptually:



Difference

Method	Receives previous result?	Produces result?
<code>thenApply()</code>	Yes	Yes
<code>thenAccept()</code>	Yes	No
<code>thenRun()</code>	No	No

11. thenCompose()

`thenCompose()` is used to **chain dependent asynchronous operations**.

Suppose:

```
Find user
↓
Find user's orders
```

The second operation depends on the result of the first.

```

CompletableFuture<String> userFuture =
    CompletableFuture.supplyAsync(
        () -> "Anurag"
    );

CompletableFuture<String> result =
    userFuture.thenCompose(
        user -> CompletableFuture.supplyAsync(
            () -> user + "'s orders"
        )
    );

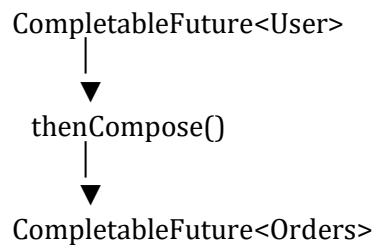
System.out.println(result.join());

```

Output:

Anurag's orders

Conceptually:



It is especially useful when the next operation itself returns a CompletableFuture.

12. thenCombine()

thenCombine() combines the results of **two independent asynchronous operations**.

Example:

```

CompletableFuture<Integer> first =
    CompletableFuture.supplyAsync(() -> 10);

CompletableFuture<Integer> second =
    CompletableFuture.supplyAsync(() -> 20);

CompletableFuture<Integer> result =
    first.thenCombine(
        second,
        (a, b) -> a + b
    );

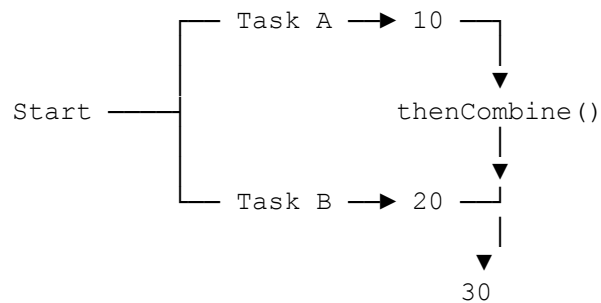
System.out.println(result.join());

```

Output:

30

Flow:

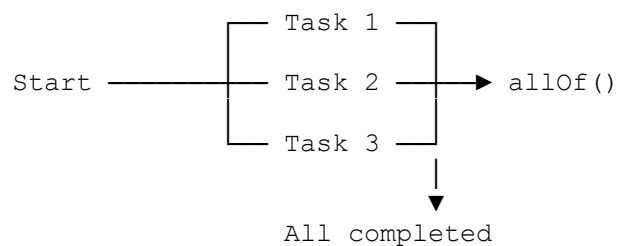


13. allOf()

`allOf()` waits for multiple `CompletableFuture` operations to complete.

```
CompletableFuture<Void> all =  
    CompletableFuture.allOf(  
        CompletableFuture.runAsync(  
            () -> System.out.println("Task 1")  
        ),  
        CompletableFuture.runAsync(  
            () -> System.out.println("Task 2")  
        ),  
        CompletableFuture.runAsync(  
            () -> System.out.println("Task 3")  
        )  
    );  
  
all.join();  
  
System.out.println("All tasks completed");
```

Conceptually:



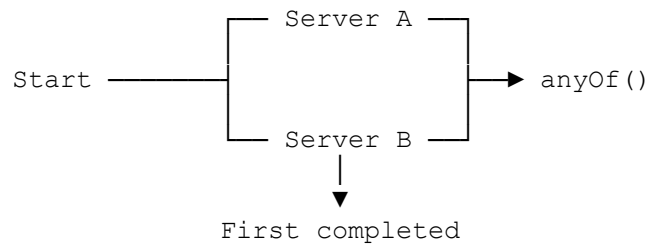
14. anyOf()

`anyOf()` completes when **any one** of the supplied futures completes.

```
CompletableFuture<Object> result =
    CompletableFuture.anyOf(
        CompletableFuture.supplyAsync(
            () -> "Server A"
        ),
        CompletableFuture.supplyAsync(
            () -> "Server B"
        )
    );

System.out.println(result.join());
```

The first completed result wins.



15. Exception Handling

One of the major benefits of `CompletableFuture` is built-in asynchronous exception handling.

`exceptionally()`

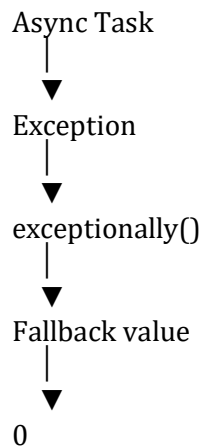
```
CompletableFuture<Integer> future =
    CompletableFuture
        .supplyAsync(() -> {
            throw new RuntimeException("Something failed");
        })
        .exceptionally(ex -> {
            System.out.println(
                "Error: " + ex.getMessage()
            );
            return 0;
        });

System.out.println(future.join());
```

Output:

Error: java.lang.RuntimeException: Something failed
0

Flow:



16. handle()

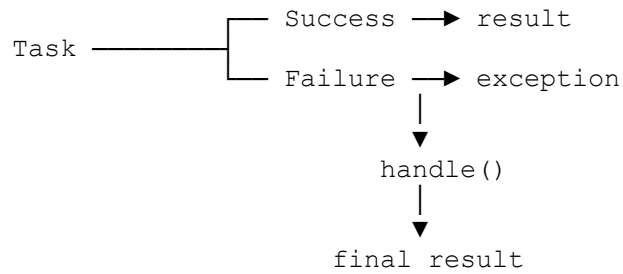
handle() receives both the result and exception.

```
CompletableFuture<Integer> future =  
    CompletableFuture.  
        .supplyAsync(() -> 10 / 0)  
        .handle((result, exception) -> {  
            if (exception != null) {  
                return 0;  
            }  
            return result;  
        });  
  
System.out.println(future.join());
```

Output:

0

Conceptually:



17. join() vs get()

Both can be used to wait for a result.

```
String result = future.join();
```

or:

```
String result = future.get();
```

The important difference is exception handling.

join()	get()
Returns result	Returns result
Throws unchecked CompletionException on exceptional completion	Throws checked InterruptedException / ExecutionException
Convenient in CompletableFuture pipelines	Traditional Future style

Although CompletableFuture supports these waiting methods, the main benefit of the API is that you can often **compose stages instead of blocking to obtain intermediate results**.

18. Complete Java Code Example

```
import java.util.concurrent.CompletableFuture;

public class CompletableFutureExample {

    public static void main(String[] args) {
```

```

CompletableFuture<String> future =
    CompletableFuture
        .supplyAsync(() -> {
            System.out.println(
                "Step 1: Loading data..."
            );
            return "Java";
        })
        .thenApply(value -> {
            System.out.println(
                "Step 2: Transforming data..."
            );
            return value.toUpperCase();
        })
        .thenApply(value -> {
            System.out.println(
                "Step 3: Adding message..."
            );
            return value + " 8";
        })
        .exceptionally(exception -> {
            System.out.println(
                "Error: " +
                exception.getMessage()
            );
            return "DEFAULT";
        });

String result = future.join();

System.out.println(
    "Final Result = " + result
);
}
}

```

Output

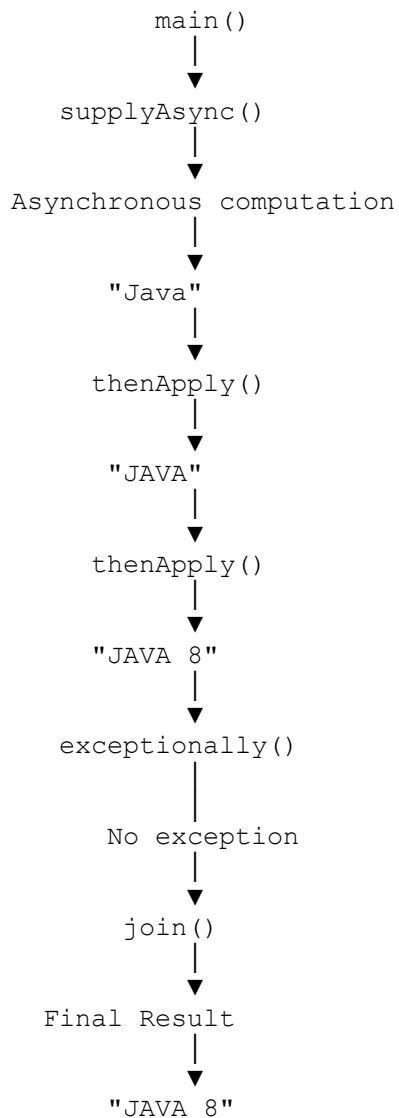
```

Step 1: Loading data...
Step 2: Transforming data...
Step 3: Adding message...
Final Result = JAVA 8

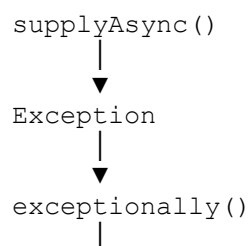
```

The exact thread executing asynchronous stages can vary, so application logic should not depend on a particular worker thread.

19. Complete Execution Flow



If an exception occurs:

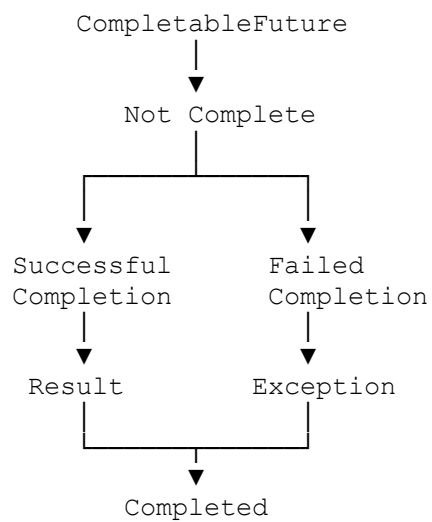


▼
Fallback value
↓
▼
join()

20. CompletableFuture State Concept

A CompletableFuture represents a computation that can move toward completion.

Conceptually:



It can also be explicitly completed:

```
CompletableFuture<String> future =  
    new CompletableFuture<>();
```

```
future.complete("Java");
```

```
System.out.println(future.join());
```

Output:

Java

21. completeExceptionally()

A future can also be explicitly completed exceptionally.

```
CompletableFuture<String> future =  
    new CompletableFuture<>();
```

```
future.completeExceptionally(  
    new RuntimeException("Failed")  
);
```

The future is then completed with an exceptional state.

22. Common Methods Summary

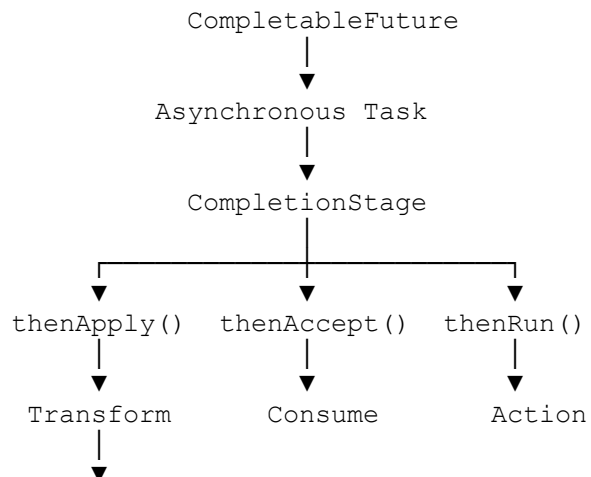
Method	Purpose
runAsync()	Execute async task without result
supplyAsync()	Execute async task with result
thenApply()	Transform result
thenAccept()	Consume result
thenRun()	Run action after completion
thenCompose()	Chain dependent futures
thenCombine()	Combine two independent results
allOf()	Wait for all supplied futures
anyOf()	Complete when any supplied future completes
exceptionally()	Recover from exception
handle()	Process result or exception
whenComplete()	Observe completion/result or exception
complete()	Explicitly complete future
completeExceptionally()	Explicitly complete with exception
join()	Obtain result, potentially waiting
get()	Obtain result, potentially waiting

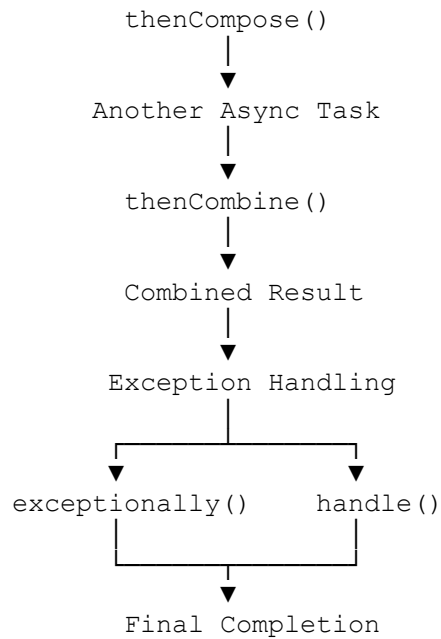
23. CompletableFuture vs Future

Feature	Future	CompletableFuture
Introduced	Java 5	Java 8
Represents async computation	Yes	Yes
Explicit completion	Limited	Yes
Chain operations	No convenient pipeline	Yes
Combine tasks	Limited	Yes
Exception pipeline	Limited	Rich API
thenApply()	No	Yes
thenCompose()	No	Yes
thenCombine()	No	Yes
allOf() / anyOf()	No	Yes
Functional-style composition	Limited	Strong support

24. Most Important Concept

The core architecture is:





One-line definition

CompletableFuture is Java 8's asynchronous computation and **CompletionStage** implementation that allows tasks to be created, explicitly completed, chained, combined, transformed, and handled for success or failure without requiring every intermediate step to block waiting for a result.

Java 8 — Base64 API

Java Version: 8

Year: 2014

Feature: Base64 API

UniqueClassName: Base64API

1. Definition

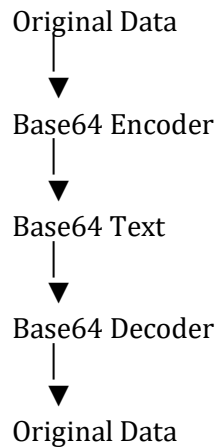
The **Base64 API** was introduced in **Java 8** to provide built-in support for **Base64 encoding and decoding** without requiring third-party libraries.

The main class is:

`java.util.Base64`

Base64 converts binary data into a text representation using a set of characters that can be conveniently transported in text-based formats.

Basic concept



For example:

"Java"
↓
"SmF2YQ=="

And decoding:

"SmF2YQ=="
↓
"Java"

Important: Base64 is an **encoding**, not encryption. Anyone who has the encoded data can decode it

2. Base64 API Package

```
import java.util.Base64;
```

The central class is:

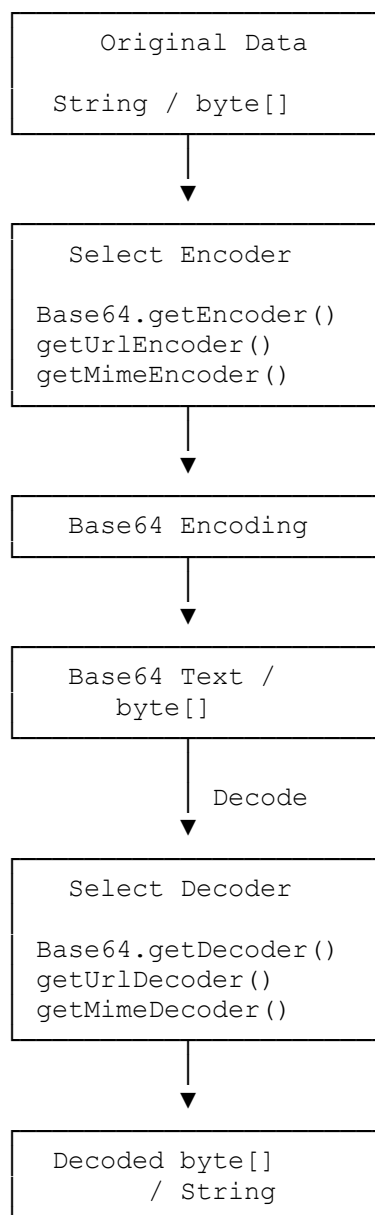
Base64

It provides three main encoder/decoder styles:

Type	Purpose
Basic	Standard Base64 encoding

URL and Filename	URL/file-name-safe Base64
MIME	MIME-compatible Base64 with line handling

3. Base64 Flowchart



4. Basic Base64 Encoding

The simplest approach is:

```
Base64.getEncoder()
```

Example:

```
import java.nio.charset.StandardCharsets;
import java.util.Base64;

String text = "Java";

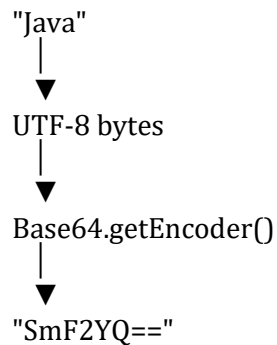
String encoded =
    Base64.getEncoder()
        .encodeToString(
            text.getBytes(StandardCharsets.UTF_8)
        );

System.out.println(encoded);
```

Output

SmF2YQ==

Flow:



5. Basic Base64 Decoding

To decode:

```
byte[] decoded =
    Base64.getDecoder().decode(encoded);
```

Then convert the bytes back to a string:

```
String result =  
    new String(decoded, StandardCharsets.UTF_8);
```

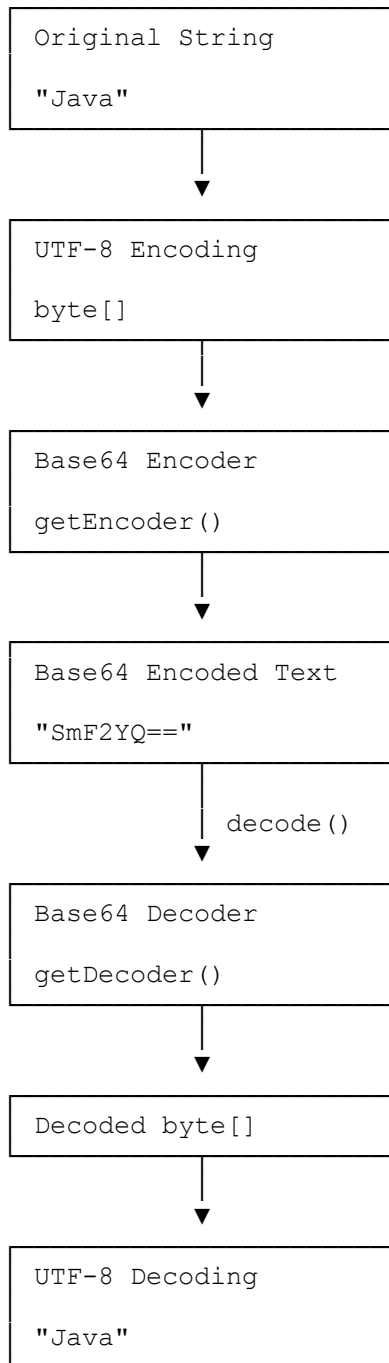
Complete example:

```
import java.nio.charset.StandardCharsets;  
import java.util.Base64;  
  
public class Base64API {  
  
    public static void main(String[] args) {  
  
        String original = "Java";  
  
        // Encode  
        String encoded =  
            Base64.getEncoder()  
                .encodeToString(  
                    original.getBytes(  
                        StandardCharsets.UTF_8  
                    )  
                );  
  
        // Decode  
        byte[] decodedBytes =  
            Base64.getDecoder()  
                .decode(encoded);  
  
        String decoded =  
            new String(  
                decodedBytes,  
                StandardCharsets.UTF_8  
            );  
  
        System.out.println(  
            "Original = " + original  
        );  
  
        System.out.println(  
            "Encoded  = " + encoded  
        );  
  
        System.out.println(  
            "Decoded  = " + decoded  
        );  
    }  
}
```

Output

```
Original = Java  
Encoded  = SmF2YQ==  
Decoded  = Java
```

6. Complete Execution Flow



7. Three Types of Base64 Encoders

Java 8 provides:

```
Base64.getEncoder()  
Base64.getUrlEncoder()  
Base64.getMimeEncoder()
```

and corresponding decoders:

```
Base64.getDecoder()  
Base64.getUrlDecoder()  
Base64.getMimeDecoder()
```

A. Basic Encoder

```
Base64.getEncoder()
```

Uses the standard Base64 alphabet.

Example:

```
String encoded =  
    Base64.getEncoder()  
        .encodeToString(  
            "Java".getBytes()  
        );
```

Output:

```
SmF2YQ==
```

B. URL Encoder

```
Base64.getUrlEncoder()
```

This variant is designed for data that needs to be safe in URLs and filenames.

It uses URL-safe alternatives for characters that have special meanings in URLs.

Example:

```
String encoded =  
    Base64.getUrlEncoder()
```

```
.encodeToString(  
    data  
);
```

Flow:

```
Binary Data  
↓  
URL Encoder  
↓  
URL-safe Base64
```

C. MIME Encoder

Base64.getMimeEncoder()

This variant is designed for MIME-style data and can insert line separators.

Example:

```
String encoded =  
    Base64.getMimeEncoder()  
        .encodeToString(  
            data  
        );
```

It is useful when dealing with MIME-oriented formats such as email content.

8. Basic vs URL vs MIME

Feature	Basic	URL/Filename	MIME
Method	getEncoder()	getUrlEncoder()	getMimeEncoder()
Standard Base64	Yes	URL-safe variant	Yes
URL-safe	No	Yes	No
MIME line handling	No	No	Yes
Typical use	General data	URLs / filenames	MIME/email-style data

9. Encoding a byte[]

Base64 is fundamentally designed to encode **bytes**, not specifically strings.

Example:

```
import java.util.Base64;

public class Base64API {

    public static void main(String[] args) {

        byte[] data = {
            10, 20, 30, 40, 50
        };

        String encoded =
            Base64.getEncoder()
                .encodeToString(data);

        byte[] decoded =
            Base64.getDecoder()
                .decode(encoded);

        System.out.println(
            "Encoded = " + encoded
        );

        System.out.println(
            "Decoded length = " + decoded.length
        );

    }
}
```

The general process is:

```
byte[]
↓
Base64 Encoder
↓
String
```

and:

```
String
↓
Base64 Decoder
↓
byte[]
```

10. encode() vs encodeToString()

The encoder provides both approaches.

encode()

Returns a byte[]:

```
byte[] encoded =  
    Base64.getEncoder()  
        .encode(data);
```

encodeToString()

Returns a String:

```
String encoded =  
    Base64.getEncoder()  
        .encodeToString(data);
```

Method	Return type
encode(byte[])	byte[]
encodeToString(byte[])	String

11. decode() Methods

The decoder can decode different input forms.

For example:

```
byte[] decoded =  
    Base64.getDecoder()  
        .decode(encodedString);
```

Or:

```
byte[] decoded =  
    Base64.getDecoder()  
        .decode(encodedBytes);
```

The result is:

`byte[]`

If the original data represented text, convert it using the correct charset:

```
String result =  
    new String(  
        decoded,  
        StandardCharsets.UTF_8  
    );
```

12. Why `StandardCharsets.UTF_8` Is Used

For text, it is better to explicitly specify the character encoding.

Prefer:

```
text.getBytes(StandardCharsets.UTF_8)
```

and:

```
new String(bytes, StandardCharsets.UTF_8)
```

rather than relying on the platform's default charset.

Complete flow:

```
String  
↓  
UTF-8  
↓  
byte[]  
↓  
Base64  
↓  
String
```

Decoding reverses it:

```
Base64 String  
↓  
Base64 decode  
↓  
byte[]  
↓  
UTF-8  
↓  
Original String
```

13. Base64 Is Not Encryption

This is an extremely important distinction.

```
Original Data
  ↓
  Base64
  ↓
Encoded Data
```

Anyone can reverse it:

```
Encoded Data
  ↓
Base64 Decode
  ↓
Original Data
```

Therefore:

```
Base64 ≠ Encryption
Base64 = Encoding
```

For example:

```
"password"
  ↓
"CGFzc3dvcnQ="
```

does **not** make the password secure.

14. Base64 with a Functional Style

Since Base64 is commonly used with Java 8 APIs, it can also be combined with lambdas and streams.

For example:

```
import java.nio.charset.StandardCharsets;
import java.util.Arrays;
import java.util.Base64;
import java.util.List;
import java.util.stream.Collectors;

public class Base64API {

    public static void main(String[] args) {

        List<String> values =
```

```

        Arrays.asList(
            "Java",
            "Python",
            "C++"
        );

        List<String> encoded =
            values.stream()
                .map(value ->
                    Base64.getEncoder()
                        .encodeToString(
                            value.getBytes(
                                StandardCharsets.UTF_8
                            )
                        )
                )
            .collect(Collectors.toList());

        System.out.println(encoded);
    }
}

```

Example output:

```
[SmF2YQ==, UHl0aG9u, Qysr]
```

Flow:

```

List<String>
↓
stream()
↓
map()
↓
Base64 Encoder
↓
collect()
↓
List<String>

```

15. Old Approach vs Java 8 Base64 API

Before Java 8, developers commonly depended on external libraries or application-specific Base64 implementations.

Java 8 provides:

```

Base64.getEncoder()
Base64.getDecoder()

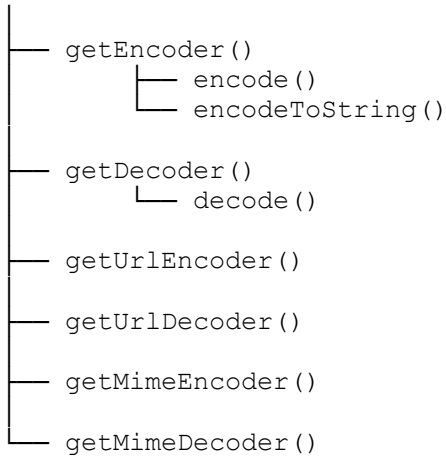
```

directly in the standard JDK.

Before Java 8	Java 8
Often external library	Built into JDK
Third-party Base64 implementations common	java.util.Base64
Different APIs depending on library	Standard Java API
Basic encoding/decoding	Basic, URL, MIME variants

16. Important Classes and Methods

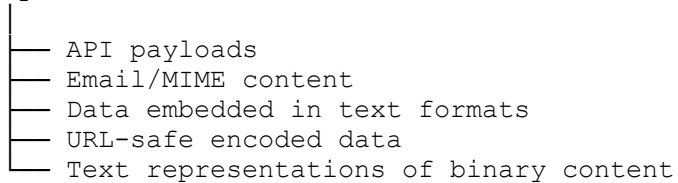
```
java.util.Base64
```



17. Real-World Conceptual Uses

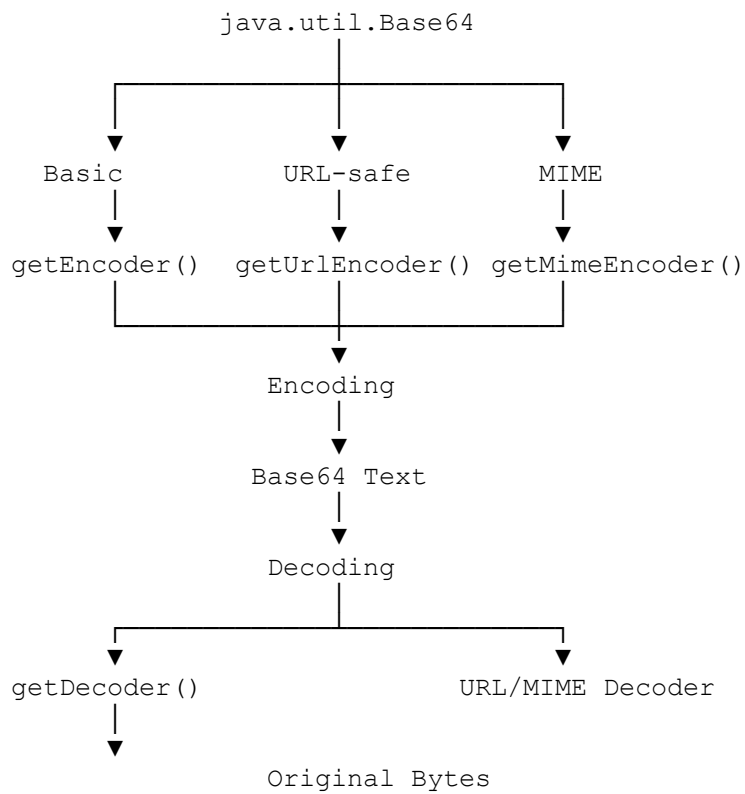
Base64 is commonly useful when binary data needs to be represented as text, for example:

```
Binary Data
```



The exact use depends on the surrounding protocol or format.

18. Key Concept



One-line definition

The Java 8 Base64 API (`java.util.Base64`) provides standard JDK support for encoding binary data into Base64 text and decoding Base64 text back into binary data, with Basic, URL/Filename-safe, and MIME variants.